



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CSA1420-Compiler Design Lab Experiments

Name: UDHAYAKUMAR U

Reg No.:192424332

EXP 1 TO 40 COMPLETED

1.) Develop a Lexical Analyzer to identify Identifiers, Constants, and Operators using a C program.

Aim

To develop a lexical analyzer that ignores redundant spaces, tabs, new lines, and comments, and identifies identifiers, constants, and operators in a C program.

Procedure

- Open the C compiler or online C compiler and create a new source file.
- Enter the lexical analyzer program and provide the input C program.
- Read the input character by character while ignoring spaces, tabs, new lines, and comments.
- Identify tokens as Identifiers, Constants, or Operators based on their type.
- Display the identified identifiers, constants, and operators as the output.

Algorithm

1. Start.
2. Read the input C program.
3. Ignore spaces, tabs, new lines, and comments.
4. Scan each character of the input.
5. If the character is a letter or `_`, read the complete token and classify it as an **Identifier**.
6. If the character is a digit, read the complete number and classify it as a **Constant**.
7. If the character is an operator symbol, classify it as an **Operator**.
8. Repeat Steps 4–7 until the end of the input.
9. Display the results.

10. Stop.

Code

```
#include <stdio.h>
```

```
#include <ctype.h>
```

```
#include <string.h>
```

```
int isOperator(char ch)
```

```
{
```

```
    char operators[] = "+-*/%=<>!&|^";
```

```
    for (int i = 0; operators[i] != '\0'; i++)
```

```
    {
```

```
        if (ch == operators[i])
```

```
            return 1;
```

```
    }
```

```
    return 0;
```

```
}
```

```
int main()
```

```
{
```

```
    char ch, buffer[100];
```

```
    int i;
```

```
printf("Paste your C program below.\n");
```

```
printf("Press Ctrl+D (Linux/Mac) or Ctrl+Z then Enter (Windows) to finish.\n\n");
```

```
while ((ch = getchar()) != EOF)
```

```
{
```

```
    if (isspace(ch))
```

```
        continue;
```

```
    // Ignore comments
```

```
    if (ch == '/')
```

```
{
```

```
        char next = getchar();
```

```
        if (next == '/')
```

```
{
```

```
            while ((ch = getchar()) != '\n' && ch != EOF);
```

```
            continue;
```

```
}
```

```
    else if (next == '*')
```

```
{
```

```
        char prev = 0;
```

```
        while ((ch = getchar()) != EOF)
```

```
{
```

```

        if (prev == '*' && ch == '/')
            break;

        prev = ch;
    }

    continue;
}

else
{
    ungetc(next, stdin);

    printf("Operator  : %c\n", ch);

    continue;
}
}

```

// Identifier

```

if (isalpha(ch) || ch == '_')
{
    i = 0;

    buffer[i++] = ch;

    while ((ch = getchar()) != EOF &&
           (isalnum(ch) || ch == '_'))
    {

```

```

        buffer[i++] = ch;
    }

    buffer[i] = '\0';
    printf("Identifier : %s\n", buffer);

    if (ch != EOF)
        ungetc(ch, stdin);
}

// Constant
else if (isdigit(ch))
{
    i = 0;
    buffer[i++] = ch;

    while ((ch = getchar()) != EOF && isdigit(ch))
    {
        buffer[i++] = ch;
    }

    buffer[i] = '\0';
    printf("Constant : %s\n", buffer);
}

```

```
        if (ch != EOF)

            ungetc(ch, stdin);

    }

    // Operator

    else if (isOperator(ch))

    {

        printf("Operator  : %c\n", ch);

    }

}


return 0;

}
```

Input

```
include<stdio.h>
```

```
int main()
```

```
{
```

```
    int a = 10;
```

```
    int b = 20;
```

```
    int c = a + b;
```

```
    return 0;
```

}

Output

Identifier : include

Operator : <

Identifier : stdio

Identifier : h

Operator : >

Identifier : int

Identifier : main

Identifier : int

Identifier : a

Operator : =

Constant : 10

Identifier : int

Identifier : b

Operator : =

Constant : 20

Identifier : int

Identifier : c

Operator : =

Identifier : a

Operator : +

Identifier : b

Identifier : return

Constant : 0

Exp : 2 Lexical Analyzer to Check Comments

Aim

To develop a lexical analyzer using Lex to identify whether the given input is a single-line comment, multi-line comment, or not a comment.

Algorithm

- ❖ Start.
- ❖ Read the input.
- ❖ Check if the input begins with `//`.
- ❖ If yes, print **Single Line Comment**.
- ❖ Else check if it begins with `/*` and ends with `*/`.
- ❖ If yes, print **Multi Line Comment**.
- ❖ Otherwise print **Not a Comment**.
- ❖ Stop.

CODE

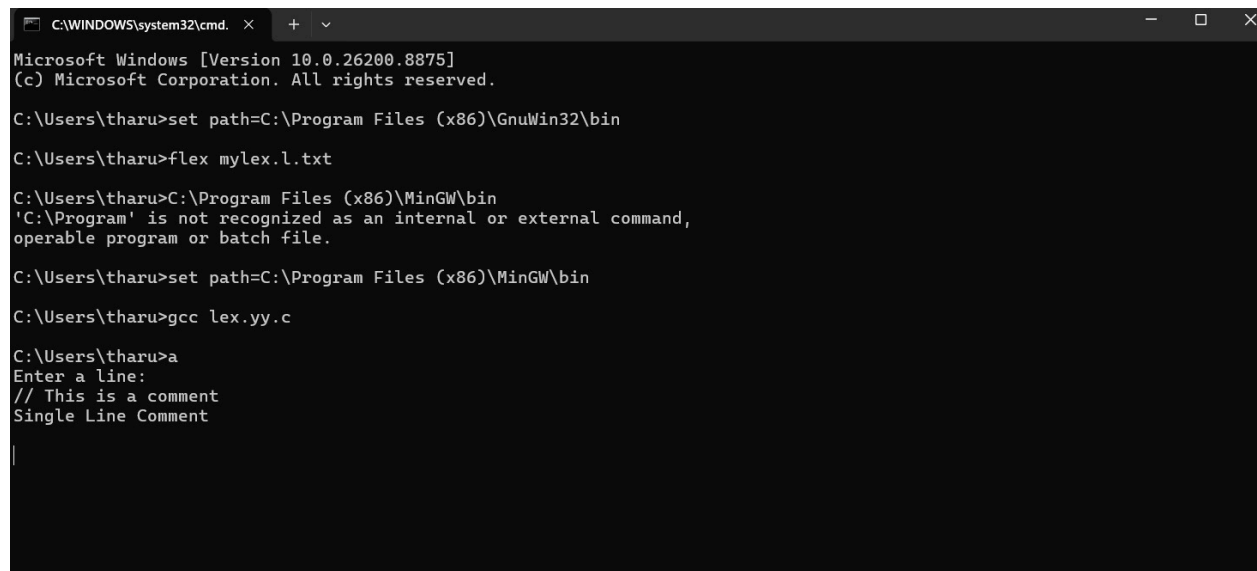
```
%{  
#include<stdio.h>  
%}  
  
%%  
  
"//".*          { printf("Single Line Comment\n"); }  
  
"/*"([^\*]|\\*+[^*/])*\*+"/" { printf("Multi Line Comment\n"); }  
  
.+              { printf("Not a Comment\n"); }
```

```
%%
```

```
int yywrap()
{
    return 1;
}
```

```
int main()
{
    printf("Enter a line:\n");
    yylex();
    return 0;
}
```

OUTPUT



```
C:\WINDOWS\system32\cmd. x + v
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

C:\Users\tharu>set path=C:\Program Files (x86)\GnuWin32\bin

C:\Users\tharu>flex mylex.l.txt

C:\Users\tharu>C:\Program Files (x86)\MinGW\bin
'C:\Program' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\tharu>set path=C:\Program Files (x86)\MinGW\bin

C:\Users\tharu>gcc lex.yy.c

C:\Users\tharu>a
Enter a line:
// This is a comment
Single Line Comment
|
```

EXP 3 : Lexical Analyzer to Ignore Spaces, Tabs, Newlines and Comments

Aim

To develop a lexical analyzer that ignores redundant spaces, tabs, newlines, and comments while recognizing identifiers and numbers.

Algorithm

- ❖ Start.
- ❖ Read the input.
- ❖ Ignore spaces.
- ❖ Ignore tabs.
- ❖ Ignore newline characters.
- ❖ Ignore comments.
- ❖ Print identifiers and numbers.
- ❖ Stop.

CODE

```
%{  
#include<stdio.h>  
%}  
  
%%  
  
"//".*          ;  
  
"/*"([^*]|\\*+[^*/])\\*+ "/" ;  
  
[ \\t\\n]+      ;
```

```
[a-zA-Z_][a-zA-Z0-9_]*    printf("IDENTIFIER : %s\n",yytext);
```

```
[0-9]+                printf("NUMBER : %s\n",yytext);
```

```
.                    printf("SYMBOL : %s\n",yytext);
```

```
%%
```

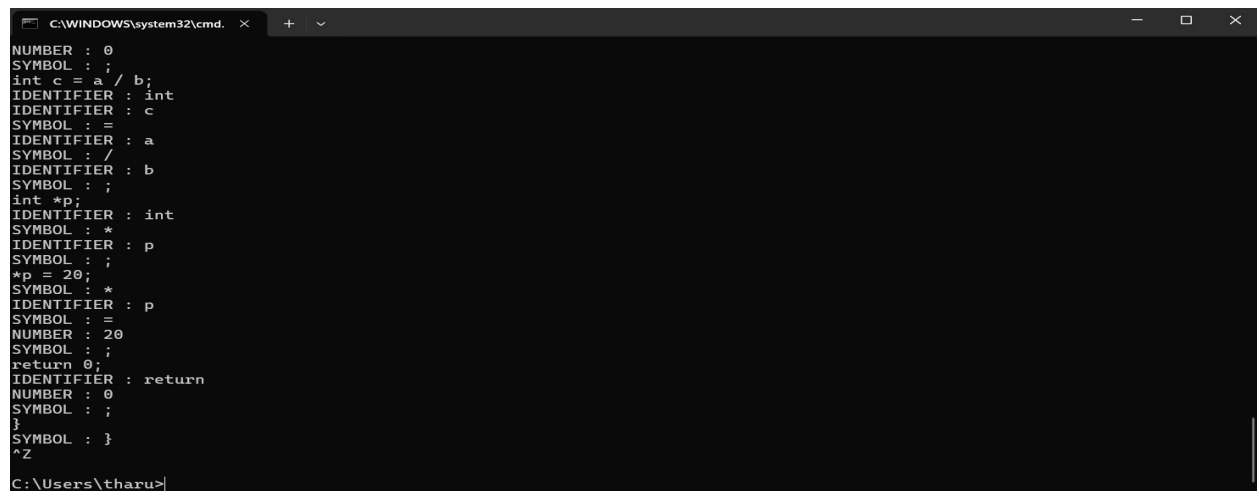
```
int yywrap()
```

```
{  
    return 1;  
}
```

```
int main()
```

```
{  
    printf("Enter Program:\n");  
    yylex();  
    return 0;  
}
```

OUTPUT



```
C:\WINDOWS\system32\cmd. x + -  
NUMBER : 0  
SYMBOL : ;  
int c = a / b;  
IDENTIFIER : int  
IDENTIFIER : c  
SYMBOL : =  
IDENTIFIER : a  
SYMBOL : /  
IDENTIFIER : b  
SYMBOL : ;  
int *p;  
IDENTIFIER : int  
SYMBOL : *  
IDENTIFIER : p  
SYMBOL : ;  
*p = 20;  
SYMBOL : *  
IDENTIFIER : p  
SYMBOL : =  
NUMBER : 20  
SYMBOL : ;  
return 0;  
IDENTIFIER : return  
NUMBER : 0  
SYMBOL : ;  
}  
SYMBOL : }  
^Z  
C:\Users\tharu>
```

EXP 4: Lexical Analyzer to Recognize Arithmetic Operators

Aim

To develop a lexical analyzer to recognize arithmetic operators (+, -, *, /).

Algorithm

- ❖ **Start.**
- ❖ **Read the input.**
- ❖ **Compare each character with**
 - ❖ +
➤
 -
- ❖ /
- ❖ **Print the corresponding operator.**
- ❖ **Stop.**

CODE

```
%{  
#include<stdio.h>  
%}  
  
%%  
  
"+"    printf("+ : Addition Operator\n");  
  
"-"    printf("- : Subtraction Operator\n");  
  
"*"    printf("* : Multiplication Operator\n");
```

```
"/"    printf("/ : Division Operator\n");
```

```
.\n    ;
```

```
%o%
```

```
int yywrap()
```

```
{  
    return 1;  
}
```

```
int main()
```

```
{  
    printf("Enter Operators:\n");  
    yylex();  
    return 0;  
}
```

OUTPUT

```
C:\WINDOWS\system32\cmd.  x  +  v  
Microsoft Windows [Version 10.0.26200.8875]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Users\tharu>C:\Program Files (x86)\GnuWin32\bin  
'C:\Program' is not recognized as an internal or external command,  
operable program or batch file.  
  
C:\Users\tharu>set path=C:\Program Files (x86)\GnuWin32\bin  
  
C:\Users\tharu>flex mylex3.l.txt  
  
C:\Users\tharu>C:\Program Files (x86)\MinGW\bin  
'C:\Program' is not recognized as an internal or external command,  
operable program or batch file.  
  
C:\Users\tharu>set path=C:\Program Files (x86)\MinGW\bin  
  
C:\Users\tharu>gcc lex.yy.c  
  
C:\Users\tharu>a  
Enter Operators:  
+  
+ : Addition Operator  
|
```

Experiment 5: Design a Lexical Analyzer to Find the Number of Whitespaces and Newline Characters

Aim

To develop a lexical analyzer using Lex (Flex) to count the number of whitespace characters, tab characters, and newline characters present in the given input.

Algorithm

- ❖ Start the program.
- ❖ Initialize three counters:

`spaces = 0`

`tabs = 0`

`newlines = 0`

- ❖ Read the input character by character using the lexical analyzer.
- ❖ If a space (' ') is encountered, increment the `spaces` counter.
- ❖ If a tab ('\t') is encountered, increment the `tabs` counter.
- ❖ If a newline ('\n') is encountered, increment the `newlines` counter.
- ❖ Ignore all other characters.
- ❖ Continue scanning until the end of the input.
- ❖ Display the total number of spaces, tabs, and newline characters.
- ❖ Stop the program.

CODE

```
#include<stdio.h>

int spaces=0;
int tabs=0;
int lines=0;
%}

%%

" "    spaces++;

"\t"   tabs++;

"\n"   lines++;

.      ;

%%

int yywrap()
{
    return 1;
}

int main()
{
    printf("Enter Text (Ctrl+Z to stop):\n");
    yylex();

    printf("\nSpaces = %d",spaces);
```

```
printf("\nTabs = %d",tabs);
printf("\nNew Lines = %d\n",lines);

return 0;
}
```

OUTPUT

```
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

C:\Users\tharu>set path=C:\Program Files (x86)\GnuWin32\bin
C:\Users\tharu>flex mylex4.l.txt
C:\Users\tharu>set path=C:\Program Files (x86)\MinGW\bin
C:\Users\tharu>gcc lex.yy.c
C:\Users\tharu>a
Enter Text (Ctrl+Z to stop):
Hello World
Welcome to Flex
^Z

Spaces = 3
Tabs = 0
New Lines = 2

C:\Users\tharu>
```

Experiment 6: Develop a Lexical Analyzer to Test Whether a Given Identifier is Valid or Not

Aim

To develop a lexical analyzer using Lex (Flex) that checks whether a given identifier is valid according to the rules of the C programming language.

Algorithm

- ❖ Start the program.
- ❖ Read the input identifier from the user.
- ❖ Check whether the first character is an alphabet (**A–Z**, **a–z**) or an underscore (**_**).
- ❖ Check whether the remaining characters contain only alphabets, digits (**0–9**), or underscores (**_**).
- ❖ If all conditions are satisfied, display "Valid Identifier".
- ❖ Otherwise, display "Invalid Identifier".
- ❖ Stop the program.

CODE

```
%{  
#include<stdio.h>  
%}  
  
%%  
  
[a-zA-Z_][a-zA-Z0-9_]*  
  
{  
    printf("%s is a Valid Identifier\n",yytext);  
}
```

[0-9][a-zA-Z0-9_]*

```
{  
    printf("%s is an Invalid Identifier\n",yytext);  
}
```

.\n ;

%%

int yywrap()

```
{  
    return 1;  
}
```

int main()

```
{  
    printf("Enter Identifier:\n");  
    yylex();  
    return 0;  
}
```

OUTPUT

```
Microsoft Windows [Version 10.0.26200.8875]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Users\tharu>set path=  
C:\Users\tharu>set path=C:\Program Files (x86)\GnuWin32\bin  
C:\Users\tharu>flex mylex5.l.txt  
C:\Users\tharu>set path=C:\Program Files (x86)\MinGW\bin  
C:\Users\tharu>gcc lex.yy.c  
C:\Users\tharu>a  
Enter Identifier:  
count  
  
C:\Users\tharu>a  
Enter Identifier:  
count  
  
C:\Users\tharu>a  
Enter Identifier:  
student123
```

Experiment 7: Write a C Program to Find FIRST() – Predictive Parser

Aim

To write a C program to find the FIRST() set of the given grammar and understand its role in predictive parsing.

Given Grammar:

$S \rightarrow AaAb \mid BbBa$

$A \rightarrow \epsilon$

$B \rightarrow \epsilon$

Algorithm

- ❖ Start the program.
- ❖ Define the grammar productions.
- ❖ Find the FIRST set for each non-terminal.
- ❖ Since $A \rightarrow \epsilon$, $\text{FIRST}(A) = \{ \epsilon \}$.
- ❖ Since $B \rightarrow \epsilon$, $\text{FIRST}(B) = \{ \epsilon \}$.
- ❖ For production $S \rightarrow AaAb$:
 - A produces ϵ , so consider the next symbol a.
 - Therefore, a is included in $\text{FIRST}(S)$.
- ❖ For production $S \rightarrow BbBa$:
 - B produces ϵ , so consider the next symbol b.
 - Therefore, b is included in $\text{FIRST}(S)$.
- ❖ Combine the results:
 - $\text{FIRST}(S) = \{ a, b \}$
- ❖ Display the FIRST sets.
- ❖ Stop the program.

CODE

```
#include <stdio.h>

int main()
{
    printf("Grammar:\n");
    printf("S -> AaAb | BbBa\n");
    printf("A ->  $\epsilon$ \n");
    printf("B ->  $\epsilon$ \n");

    // FIRST sets
    printf("\nFIRST(A) = {  $\epsilon$  }\n");
    printf("FIRST(B) = {  $\epsilon$  }\n");

    printf("\nFinding FIRST(S):\n");

    printf("Production 1: S -> AaAb\n");
    printf("A produces  $\epsilon$ , so move to next symbol 'a'.\n");
    printf("Therefore, 'a' belongs to FIRST(S).\n");

    printf("\nProduction 2: S -> BbBa\n");
    printf("B produces  $\epsilon$ , so move to next symbol 'b'.\n");
    printf("Therefore, 'b' belongs to FIRST(S).\n");

    printf("\nFIRST(S) = { a, b }\n");

    return 0;
}
```

OUTPUT

Output

Grammar:

$S \rightarrow AaAb \mid BbBa$

$A \rightarrow \epsilon$

$B \rightarrow \epsilon$

$\text{FIRST}(A) = \{ \epsilon \}$

$\text{FIRST}(B) = \{ \epsilon \}$

Finding $\text{FIRST}(S)$:

Production 1: $S \rightarrow AaAb$

A produces ϵ , so move to next symbol 'a'.

Therefore, 'a' belongs to $\text{FIRST}(S)$.

Production 2: $S \rightarrow BbBa$

B produces ϵ , so move to next symbol 'b'.

Therefore, 'b' belongs to $\text{FIRST}(S)$.

$\text{FIRST}(S) = \{ a, b \}$

=== Code Execution Successful ===

Experiment 8: Write a C program to find FOLLOW() - predictive parser for the given grammar

Aim

To write a C program to find the FOLLOW set of the non-terminals in a context-free grammar.

Algorithm

- ❖ Start the program.
- ❖ Read the grammar productions.
- ❖ Identify all non-terminals.
- ❖ Add \$ to the FOLLOW set of the start symbol.
- ❖ Apply FOLLOW set rules for each production.
- ❖ Continue until no new symbols are added.
- ❖ Display the FOLLOW set of each non-terminal.
- ❖ Stop.

Code

```
#include <stdio.h>

int main()
{
    printf("Grammar:\n");
    printf("S -> AaAb | BbBa\n");
    printf("A -> e\n");
    printf("B -> e\n");

    printf("\nFOLLOW Sets:\n");
    printf("FOLLOW(S) = { $ }\n");
```

```
printf("FOLLOW(A) = { a, b }\n");  
printf("FOLLOW(B) = { b, a }\n");  
  
return 0;  
}
```

Output

Output

Grammar :

$S \rightarrow AaAb \mid BbBa$

$A \rightarrow \epsilon$

$B \rightarrow \epsilon$

FOLLOW Sets:

$FOLLOW(S) = \{ \$ \}$

$FOLLOW(A) = \{ a, b \}$

$FOLLOW(B) = \{ b, a \}$

=== Code Execution Successful ===

Experiment 9 : Implement a C program to eliminate left recursion from a given CFG.

Aim

To write a C program to eliminate left recursion from a given context-free grammar.

Algorithm

- ❖ Start the program.
- ❖ Read the grammar productions.
- ❖ Check whether the production contains immediate left recursion.
- ❖ If left recursion exists, divide the productions into recursive and non-recursive parts.
- ❖ Create a new non-terminal (A').
- ❖ Rewrite the grammar as:
 - ❖ $A \rightarrow \beta A'$
 - ❖ $A' \rightarrow \alpha A' \mid \epsilon$
- ❖ Display the transformed grammar.
- ❖ Stop.

Code

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    printf("Given Grammar:\n");
```

```
    printf("S -> (L) | a\n");
```

```
    printf("L -> L,S | S\n");
```

```
printf("\nGrammar after Eliminating Left Recursion:\n");
printf("S -> (L) | a\n");
printf("L -> SL\n");
printf("L' -> ,SL' | e\n");

return 0;
}
```

Output

Output

Given Grammar:

S -> (L) | a

L -> L,S | S

Grammar after Eliminating Left Recursion:

S -> (L) | a

L -> SL'

L' -> ,SL' | e

=== Code Execution Successful ===

Experiment 10 : Implement a C program to eliminate left factoring from a given CFG.

Aim

To write a C program to perform left factoring on a given grammar.

Algorithms

- ❖ Start the program.
- ❖ Read the grammar productions.
- ❖ Find productions having a common prefix.
- ❖ Remove the common prefix.
- ❖ Create a new non-terminal.
- ❖ Rewrite the productions using left factoring.
- ❖ Display the transformed grammar.
- ❖ Stop.

Code

```
#include <stdio.h>

int main()
{
    printf("Given Grammar:\n");
    printf("S -> iEtS | iEtSeS | a\n");
    printf("E -> b\n");

    printf("\nGrammar after Left Factoring:\n");
    printf("S -> iEtSS' | a\n");
    printf("S' -> eS | e\n");
    printf("E -> b\n");
}
```

```
    return 0;  
}
```

Output

Output

Given Grammar:

$S \rightarrow iEtS \mid iEtSeS \mid a$

$E \rightarrow b$

Grammar after Left Factoring:

$S \rightarrow iEtSS' \mid a$

$S' \rightarrow eS \mid \epsilon$

$E \rightarrow b$

=== Code Execution Successful ===

Experiment 11 : Implement a C program to perform symbol table operations.

Aim

To implement a C program to perform basic symbol table operations such as insertion, searching, and displaying identifiers.

Algorithm

- ❖ Start the program.
- ❖ Create a structure to store identifier name and data type.
- ❖ Display the menu.
- ❖ Read the user's choice.
- ❖ If the choice is **Insert**, read the identifier name and data type and store them in the symbol table.
- ❖ If the choice is **Display**, print all the entries in the symbol table.
- ❖ If the choice is **Search**, read the identifier name and search for it in the table.
- ❖ Display whether the identifier is found or not.
- ❖ Repeat the process until the user chooses to exit.
- ❖ Stop the program.

Code

```
#include <stdio.h>
#include <string.h>

struct symbol
{
    char name[20];
    char type[20];
};
```

```
struct symbol s[20];  
int count = 0;
```

```
void insert()  
{  
    printf("Enter Identifier Name: ");  
    scanf("%s", s[count].name);  
  
    printf("Enter Data Type: ");  
    scanf("%s", s[count].type);  
  
    count++;  
}
```

```
void display()  
{  
    int i;  
    printf("\nSYMBOL TABLE\n");  
    printf("-----\n");  
    printf("Identifier\tType\n");  
  
    for(i = 0; i < count; i++)  
        printf("%s\t\t%s\n", s[i].name, s[i].type);  
}
```

```
void search()  
{  
    char key[20];  
    int i, flag = 0;  
  
    printf("Enter Identifier to Search: ");
```

```
scanf("%s", key);
```

```
for(i = 0; i < count; i++)  
{  
    if(strcmp(key, s[i].name) == 0)  
    {  
        printf("Identifier Found\n");  
        flag = 1;  
        break;  
    }  
}
```

```
if(flag == 0)  
    printf("Identifier Not Found\n");  
}
```

```
int main()  
{  
    int ch;
```

```
while(1)  
{  
    printf("\n1.Insert\n2.Display\n3.Search\n4.Exit\n");  
    printf("Enter Choice: ");  
    scanf("%d", &ch);
```

```
switch(ch)  
{  
    case 1: insert(); break;  
    case 2: display(); break;  
    case 3: search(); break;  
    case 4: return 0;
```

```

        default: printf("Invalid Choice\n");
    }
}
}

```

Output

```

1.Insert
2.Display
3.Search
4.Exit
Enter Choice: 1
Enter Identifier Name: x
Enter Data Type: int

1.Insert
2.Display
3.Search
4.Exit
Enter Choice: 1
Enter Identifier Name: y
Enter Data Type: float

1.Insert
2.Display
3.Search
4.Exit
Enter Choice: 2

SYMBOL TABLE
-----
Identifier  Type
x          int
y          float

1.Insert
2.Display
3.Search
4.Exit
Enter Choice: 3
Enter Identifier to Search: x
Identifier Found

```

Experiment 12 : Write a C program to construct recursive descent parsing for the given grammar

Aim

To implement a C program for Recursive Descent Parsing for the grammar:

$$\begin{aligned} E &\rightarrow TE' \\ E' &\rightarrow +TE' \mid \varepsilon \\ T &\rightarrow FT' \\ T' &\rightarrow *FT' \mid \varepsilon \\ F &\rightarrow (E) \mid i \end{aligned}$$

Algorithm

- ❖ Start the program.
- ❖ Read the input expression.
- ❖ Call the function $E()$.
- ❖ $E()$ calls $T()$ and $E'()$.
- ❖ $T()$ calls $F()$ and $T'()$.
- ❖ $F()$ checks for an identifier (i) or an expression enclosed in parentheses.
- ❖ If the entire input string is parsed successfully, display String Accepted.
- ❖ Otherwise, display String Rejected.
- ❖ Stop.

Code

```
#include<stdio.h>
#include<string.h>

char input[20];
```

```
int i = 0;
```

```
void E();  
void EP();  
void T();  
void TP();  
void F();
```

```
void E()  
{  
    T();  
    EP();  
}
```

```
void EP()  
{  
    if(input[i] == '+')  
    {  
        i++;  
        T();  
        EP();  
    }  
}
```

```
void T()  
{  
    F();  
    TP();  
}
```

```
void TP()  
{
```

```
    if(input[i] == '*')
    {
        i++;
        F();
        TP();
    }
}
```

```
void F()
{
    if(input[i] == 'i')
    {
        i++;
    }
    else if(input[i] == '(')
    {
        i++;
        E();
        if(input[i] == ')')
            i++;
    }
}
```

```
int main()
{
    printf("Enter Expression: ");
    scanf("%s", input);

    E();

    if(input[i] == '\0')
        printf("String Accepted");
}
```

```
else
    printf("String Rejected");

return 0;
}
```

Output

Output

```
Enter Expression: i+i*i
String Accepted
```

```
=== Code Execution Successful ===
```

Experiment 13: All languages have Grammar. When people frame a sentence we usually say whether the sentence is framed as per the rules of the Grammar or Not. Similarly use the same ideology, implement either Top Down parsing technique or Bottom Up Parsing technique to check whether the given input string is satisfying the grammar or not.

Aim

To implement a C program using Top Down Parsing to verify whether the given input string satisfies the grammar.

Algorithm

- ❖ Start the program.
- ❖ Read the input string.
- ❖ Begin parsing from the start symbol.
- ❖ Match each input symbol according to the grammar.
- ❖ If all symbols are matched successfully, accept the string.
- ❖ Otherwise, reject the string.
- ❖ Stop.

Code

```
#include<stdio.h>
```

```
char str[20];
```

```
int pos = 0;
```

```
void S()
```

```
{
```

```
    if(str[pos] == 'a')
    {
        pos++;

        if(str[pos] == 'b')
            pos++;
    }
}

int main()
{
    printf("Enter String: ");
    scanf("%s", str);

    S();

    if(str[pos] == '\0')
        printf("String Accepted");
    else
        printf("String Rejected");
```

```
    return 0;  
}
```

Output

Output

Enter String: ab

String Accepted

=== Code Execution Successful ===

Experiment 14: The main function of the Intermediate code generation is producing three address code statements for a given input expression. The three address codes help in determining the sequence in which operations are actioned by the compiler. The key work of Intermediate code generators is to simplify the process of Code generators. Write a C Program to Generate the Three address code representation for the given input statement.

Aim

To implement a C program to generate Three Address Code for a given arithmetic expression.

Algorithm

- ❖ Start the program.
- ❖ Read the arithmetic expression.
- ❖ Scan the expression from left to right.
- ❖ Identify arithmetic operators.
- ❖ Generate temporary variables for each operation.
- ❖ Print the Three Address Code.
- ❖ Stop.

Code

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    char exp[20];
```

```
int i = 0;
```

```
char temp = 'A';
```

```
printf("Enter Expression: ");
```

```
scanf("%s", exp);
```

```
while(exp[i] != '\0')
```

```
{
```

```
    if(exp[i] == '+' || exp[i] == '-' ||
```

```
        exp[i] == '*' || exp[i] == '/')
```

```
    {
```

```
        printf("%c = %c %c %c\n",
```

```
            temp,
```

```
            exp[i-1],
```

```
            exp[i],
```

```
            exp[i+1]);
```

```
        exp[i+1] = temp;
```

```
        temp++;
```

```
    }
```

```
        i++;  
    }  
  
    return 0;  
}
```

Output

Output

```
Enter Expression: a=b*c  
A = b * c
```

```
=== Code Execution Successful ===
```

Experiment 15 : Write a C program for implementing a Lexical Analyzer to Scan and Count the number of characters, words, and lines in a file

Aim

To implement a C program that counts the number of characters, words, and lines present in a file.

Algorithm

- Start the program.
- Open the input file in read mode.
- Initialize character, word, and line counters.
- Read each character until the end of the file.
- Increment the character count for every character read.
- Increment the word count when a space or newline is encountered.
- Increment the line count whenever a newline character is found.
- Display the total number of characters, words, and lines.
- Close the file.
- Stop.

Code

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
int main()
```

```
{
```

```
    FILE *fp;
```

```
char ch;

int characters = 0, words = 1, lines = 1;

fp = fopen("sample.txt", "r");

if(fp == NULL)
{
    printf("File Not Found");
    return 0;
}

while((ch = fgetc(fp)) != EOF)
{
    characters++;

    if(ch == ' ' || ch == '\t')
        words++;

    if(ch == '\n')
    {
```

```
        lines++;

        words++;

    }

}

fclose(fp);

printf("Characters = %d\n", characters);
printf("Words = %d\n", words);
printf("Lines = %d\n", lines);

return 0;

}
```

Output

Experiment 16 : Write a C program to implement the back end of the compiler.

Aim

To write a C program that simulates the back end of a compiler by generating simple target code (assembly-like instructions) from arithmetic expressions.

Algorithm

- ❖ Start the program.
- ❖ Read an arithmetic expression.
- ❖ Scan the expression character by character.
- ❖ Identify operands and operators.
- ❖ Generate assembly-like instructions for each operation.
- ❖ Display the generated target code.
- ❖ Stop the program.

Code

```
#include <stdio.h>
#include <string.h>

int main()
{
    char expr[50];
    int i;

    printf("Enter Expression (Example: A=B+C): ");
    scanf("%s", expr);

    printf("\nGenerated Target Code:\n");
```

```

for(i = 2; i < strlen(expr); i++)
{
    if(expr[i] == '+')
    {
        printf("MOV R0,%c\n", expr[i-1]);
        printf("ADD R0,%c\n", expr[i+1]);
        printf("MOV %c,R0\n", expr[0]);
    }
    else if(expr[i] == '-')
    {
        printf("MOV R0,%c\n", expr[i-1]);
        printf("SUB R0,%c\n", expr[i+1]);
        printf("MOV %c,R0\n", expr[0]);
    }
    else if(expr[i] == '*')
    {
        printf("MOV R0,%c\n", expr[i-1]);
        printf("MUL R0,%c\n", expr[i+1]);
        printf("MOV %c,R0\n", expr[0]);
    }
    else if(expr[i] == '/')
    {
        printf("MOV R0,%c\n", expr[i-1]);
        printf("DIV R0,%c\n", expr[i+1]);
        printf("MOV %c,R0\n", expr[0]);
    }
}

return 0;
}

```

INPUT

A=B+C

OUTPUT

```
Enter Expression (Example: A=B+C): A=B+C
```

```
Generated Target Code:
```

```
MOV R0,B
```

```
ADD R0,C
```

```
MOV A,R0
```

```
=== Code Execution Successful ===
```

Experiment 17 : Compute LEADING() – Operator Precedence Parser

Aim

To write a C program to compute the LEADING() set for the given grammar.

Grammar

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow (E) \mid id$$

Algorithm

- ❖ Start the program.
- ❖ Define the grammar productions.
- ❖ Find the first terminal appearing in each production.
- ❖ Store the terminal in the LEADING set.
- ❖ Display the LEADING set.
- ❖ Stop.

Code

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    printf("LEADING Sets\n");
```

```
    printf("LEADING(E) = { (, id }\n");
```

```
    printf("LEADING(T) = { (, id }\n");  
    printf("LEADING(F) = { (, id }\n");  
    return 0;  
}
```

OUTPUT

```
LEADING Sets  
LEADING(E) = { (, id }  
LEADING(T) = { (, id }  
LEADING(F) = { (, id }  
  
=== Code Execution Successful ===
```

Experiment 18 : Compute TRAILING() – Operator Precedence Parser

Aim

To write a C program to compute the **TRAILING()** set for the given grammar.

Grammar:

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow (E) \mid id$$

Algorithm

1. Start the program.
2. Define the grammar.
3. Find the last terminal of every production.
4. Store the terminal in the TRAILING set.
5. Display the result.
6. Stop.

Code

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    printf("TRAILING Sets\n");
```

```
printf("TRAILING(E) = { }, id }\n");  
printf("TRAILING(T) = { }, id }\n");  
printf("TRAILING(F) = { }, id }\n");  
return 0;  
}
```

OUTPUT

```
TRAILING Sets
```

```
TRAILING(E) = { }, id }
```

```
TRAILING(T) = { }, id }
```

```
TRAILING(F) = { }, id }
```

```
=== Code Execution Successful ===
```

Experiment 19: LEX Program to Count Characters, Words and Lines

Aim

To write a LEX specification that ignores spaces, tabs, newlines, and comments while counting the number of characters, words, and lines in a C source file.

Algorithm

1. Start the program.
2. Read the input C source file.
3. Ignore comments.
4. Ignore redundant spaces and tabs.
5. Count characters.
6. Count words.
7. Count lines.
8. Display the counts.
9. Stop.

Code

```
%{  
  
#include <stdio.h>  
  
int char_count = 0, word_count = 0, line_count = 0;  
  
%}  
  
%option noyywrap  
  
%%  
  
"/*"([^\]|\\*+[^/])*\*+/" { /* Ignore multi-line comments */ }  
  
"/".* { /* Ignore single-line comments */ }
```

```
\n                { line_count++; char_count++; }  
[ \t]+            { char_count += yyleng; }  
[a-zA-Z0-9_]{1,31}    { word_count++; char_count += yyleng; }  
.                    { char_count++; }
```

```
%%
```

```
int main(int argc, char **argv) {  
    if(argc > 1) {  
        FILE *fp = fopen(argv[1], "r");  
        if(fp) yyin = fp;  
    }  
    yylex();  
    printf("Lines: %d\nWords: %d\nCharacters: %d\n", line_count,  
word_count, char_count);  
    return 0;  
}
```

Sample Input:

```
#include <stdio.h>

int main()

{

int number1, number2, sum;

printf("Enter two integers: ");

scanf("%d %d", &number1, &number2);

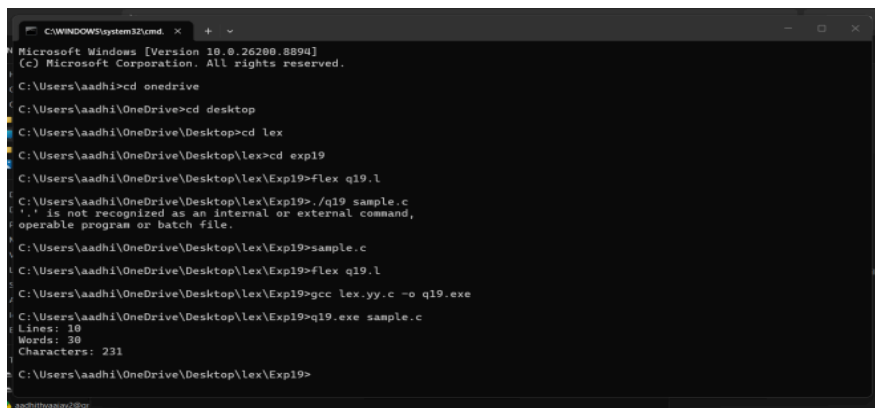
sum = number1 + number2;

printf("%d + %d = %d", number1, number2, sum);

return 0;

}
```

Output:



```
C:\WINDOWS\system32\cmd. X + ~
Microsoft Windows [Version 10.0.26200.8894]
(c) Microsoft Corporation. All rights reserved.

C:\Users\aadhi>cd onedrive
C:\Users\aadhi\OneDrive>cd desktop
C:\Users\aadhi\OneDrive\Desktop>cd lex
C:\Users\aadhi\OneDrive\Desktop\lex>cd exp19
C:\Users\aadhi\OneDrive\Desktop\lex\Exp19>flex q19.l
C:\Users\aadhi\OneDrive\Desktop\lex\Exp19>./q19 sample.c
'.' is not recognized as an internal or external command,
operable program or batch file.
C:\Users\aadhi\OneDrive\Desktop\lex\Exp19>sample.c
C:\Users\aadhi\OneDrive\Desktop\lex\Exp19>flex q19.l
C:\Users\aadhi\OneDrive\Desktop\lex\Exp19>gcc lex.yy.c -o q19.exe
C:\Users\aadhi\OneDrive\Desktop\lex\Exp19>q19.exe sample.c
Lines: 10
Words: 30
Characters: 231
C:\Users\aadhi\OneDrive\Desktop\lex\Exp19>
```

20. Write a LEX program to print all the constants in the given C source program file.

Input Source Program: (sample.c)

```
#define PI 3.14

#include<stdio.h>

#include<conio.h>

void main()

{

int a,b,c = 30;

printf("hello");

}
```

Write a LEX program to count the number of Macros defined and header files included in the C program.

Input Source Program: (sample.c)

```
#define PI 3.14

#include<stdio.h>

#include<conio.h>

void main()

{

int a,b,c = 30;

printf("hello");

}
```

Aim:

To write a LEX program to identify constants and count macros and header files in a C program.

Short Procedure:

- Define patterns for constants.
- Define patterns for macros and header files.
- Read the input C program.
- Count and print matched tokens.
- Display the total count.

Code:

Part A: All constant

```
%{  
#include <stdio.h>  
%}  
%option noyywrap  
%%  
\"[^\"]*\"          { printf("String Constant: %s\\n", yytext); }  
'[^']*'           { printf("Character Constant: %s\\n", yytext); }  
[0-9]+\.[0-9]+    { printf("Floating-Point Constant: %s\\n", yytext); }  
[0-9]+            { printf("Integer Constant: %s\\n", yytext); }
```

```
.\n          { /* Ignore other tokens */ }
```

```
%%
```

```
int main(int argc, char **argv) {  
    if(argc > 1) {  
        FILE *f = fopen(argv[1], "r");  
        if(f) yyin = f;  
    }  
    yylex();  
    return 0;  
}
```

Part B: Count Macros and Header files

```
%{  
  
#include <stdio.h>  
  
int macro_count = 0;  
  
int header_count = 0;  
  
%}
```

```
%option noyywrap
```

```
%%
```

```
#define[ \t]+[A-Za-z_][A-Za-z0-9_]*.* { macro_count++; }
```

```
#include[ \t]*[<"][^>"]+[>"] { header_count++; }
```

```
.\n { /* Ignore rest */ }
```

```
%%
```

```
int main(int argc, char **argv) {
```

```
    if(argc > 1) {
```

```
        FILE *f = fopen(argv[1], "r");
```

```
        if(f) yyin = f;
```

```
    }
```

```
    yylex();
```

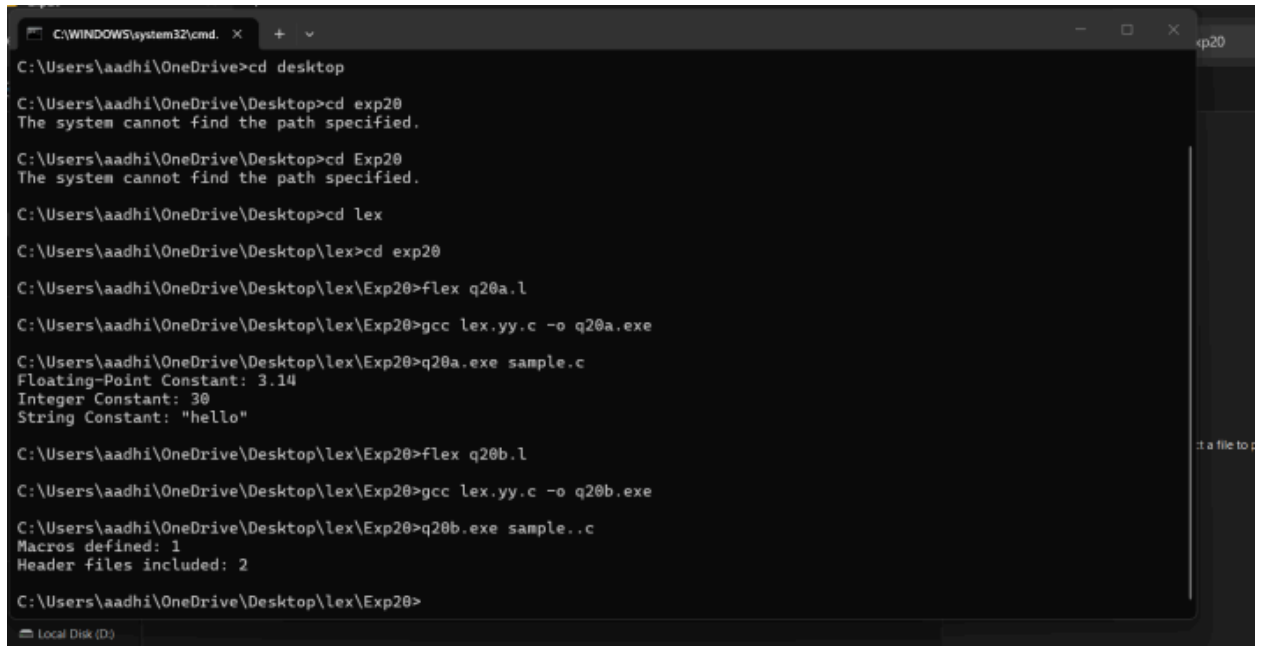
```
    printf("Macros defined: %d\n", macro_count);
```

```
    printf("Header files included: %d\n", header_count);
```

```
    return 0;
```

```
}
```

Output:



```
C:\WINDOWS\system32\cmd. x + v
C:\Users\aadhi\OneDrive>cd desktop
C:\Users\aadhi\OneDrive\Desktop>cd exp20
The system cannot find the path specified.
C:\Users\aadhi\OneDrive\Desktop>cd Exp20
The system cannot find the path specified.
C:\Users\aadhi\OneDrive\Desktop>cd lex
C:\Users\aadhi\OneDrive\Desktop\lex>cd exp20
C:\Users\aadhi\OneDrive\Desktop\lex\Exp20>flex q20a.l
C:\Users\aadhi\OneDrive\Desktop\lex\Exp20>gcc lex.yy.c -o q20a.exe
C:\Users\aadhi\OneDrive\Desktop\lex\Exp20>q20a.exe sample.c
Floating-Point Constant: 3.14
Integer Constant: 30
String Constant: "hello"
C:\Users\aadhi\OneDrive\Desktop\lex\Exp20>flex q20b.l
C:\Users\aadhi\OneDrive\Desktop\lex\Exp20>gcc lex.yy.c -o q20b.exe
C:\Users\aadhi\OneDrive\Desktop\lex\Exp20>q20b.exe sample..c
Macros defined: 1
Header files included: 2
C:\Users\aadhi\OneDrive\Desktop\lex\Exp20>
```

21. In a class, an English teacher was teaching the vowels and consonants to the students. She says “Vowel sounds allow the air to flow freely, causing the chin to drop noticeably, whilst consonant sounds are produced by restricting the air flow”. As a class activity the students are asked to identify the vowels and consonants in the given word/sentence and count the number of elements in each. Write an algorithm to help the student to count the number of vowels in the given sentence.

Aim:

To write an algorithm/program to count the number of vowels in a given sentence.

Short Procedure:

1. Read the input sentence.
2. Traverse each character.
3. Check whether it is a vowel.
4. Increment the vowel count.
5. Display the total vowels.

Code:

```
#include <stdio.h>

#include <ctype.h>

int main() {

    char str[] = "Compiler Design Laboratory";

    int count = 0;

    for(int i = 0; str[i] != '\0'; i++) {

        char ch = tolower(str[i]);

        if(ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u') count++;

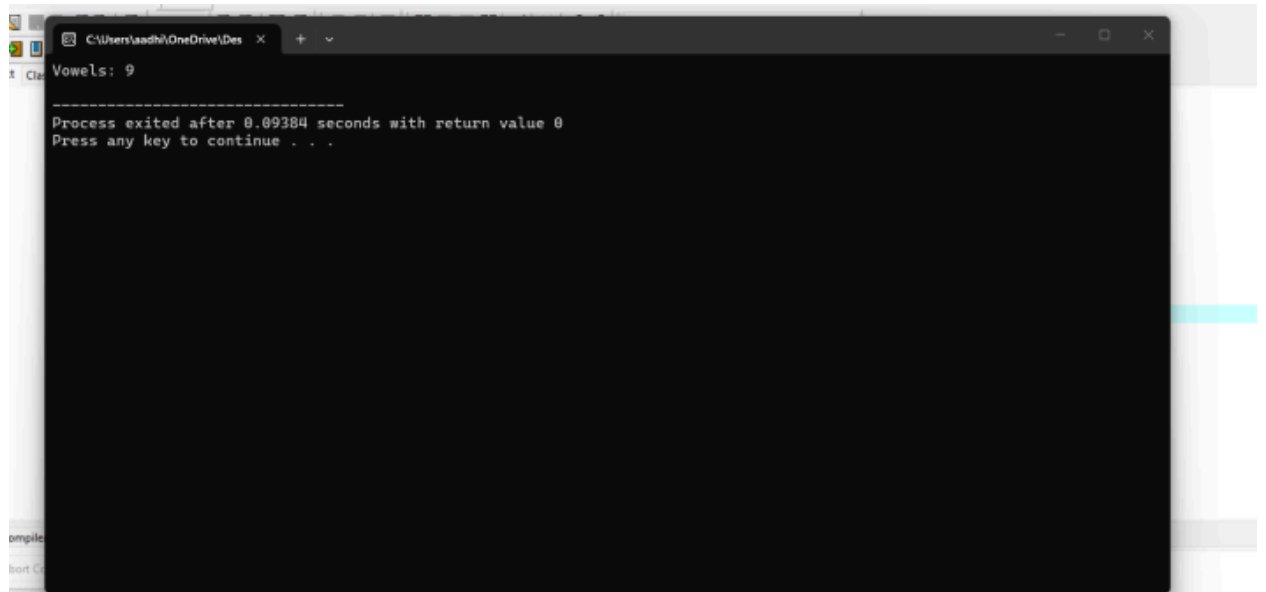
    }

    printf("Vowels: %d\n", count);

    return 0;

}
```

Output:



A screenshot of a Windows command prompt window. The title bar shows the file path "C:\Users\aadhi\OneDrive\Desktop". The window content displays the following text: "Vowels: 9", followed by a line of dashes "-----", then "Process exited after 0.09384 seconds with return value 0", and finally "Press any key to continue . . .". The window is set against a black background with white text.

```
C:\Users\aadhi\OneDrive\Desktop >
Vowels: 9
-----
Process exited after 0.09384 seconds with return value 0
Press any key to continue . . .
```

22. Write a LEX program to print all HTML tags in the input file.

Input Source Program: (sample.html)

<html>

<body>

<h1>My First Heading</h1>

<p>My first paragraph.</p>

</body>

</html>

Aim:

To write a LEX program to identify and print HTML tags from an HTML file.

Short Procedure:

- 1. Read the HTML file.
- 2. Match HTML tag patterns.
- 3. Print each identified tag.
- 4. Ignore normal text.
- 5. Display all HTML tags.

Code:

```
%{  
  
#include <stdio.h>  
  
%}  
  
%option noyywrap  
  
%%  
  
"<"[^>]+">" { printf("HTML Tag: %s\n", yytext); }  
  
.|\\n      { /* Ignore non-tag text */ }  
  
  
%%  
  
int main(int argc, char **argv) {  
  
    if (argc > 1) {  
  
        FILE *fp = fopen(argv[1], "r");  
  
        if (fp) yyin = fp;  
  
    }  
  
    yylex();  
  
    return 0;  
}  
int yywrap()  
{  
  
    return 1;  
}
```

Sample input:

```
<html>

<body>

<h1>My First Heading</h1>

<p>My first paragraph.</p>

</body>

</html>
```

Output:

```
C:\Users\aadhi\OneDrive\Desktop\lex>cd exp22
C:\Users\aadhi\OneDrive\Desktop\lex\Exp22>flex exp22.l
C:\Users\aadhi\OneDrive\Desktop\lex\Exp22>gcc lex.yy.c -o exp22 -lfl
c:/flex/windows/gcc/bin/./lib/gcc/mingw32/4.7.2/../../../../mingw32/bin/ld.exe: cannot find -lfl
collect2.exe: error: ld returned 1 exit status
C:\Users\aadhi\OneDrive\Desktop\lex\Exp22>lex.yy.c
C:\Users\aadhi\OneDrive\Desktop\lex\Exp22>gcc lex.yy.c -o htmltag -lfl
c:/flex/windows/gcc/bin/./lib/gcc/mingw32/4.7.2/../../../../mingw32/bin/ld.exe: cannot find -lfl
collect2.exe: error: ld returned 1 exit status
C:\Users\aadhi\OneDrive\Desktop\lex\Exp22>
C:\Users\aadhi\OneDrive\Desktop\lex\Exp22>gcc lex.yy.c -o exp22.l
C:\Users\aadhi\OneDrive\Desktop\lex\Exp22>exp22.l sample.html
HTML Tag: <html>
HTML Tag: <body>
HTML Tag: <h1>
HTML Tag: </h1>
HTML Tag: <p>
HTML Tag: </p>
HTML Tag: </body>
HTML Tag: </html>
C:\Users\aadhi\OneDrive\Desktop\lex\Exp22>
```

HTML Tag Parsing

Windows.

Try this instead

23. Write a LEX program which adds line numbers to the given C program file and display

the same in the standard output.

Input Source Program: (sample.c)

```
#define PI 3.14
```

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main()
```

```
{
```

```
int a,b,c = 30;
```

```
printf("hello");
```

```
}
```

Aim:

To write a LEX program to display a C program with line numbers.

Short Procedure:

- 1. Read the C source file.
- 2. Initialize the line counter.
- 3. Print each line with its line number.
- 4. Increment the counter.
- 5. Display the numbered program.

Code:

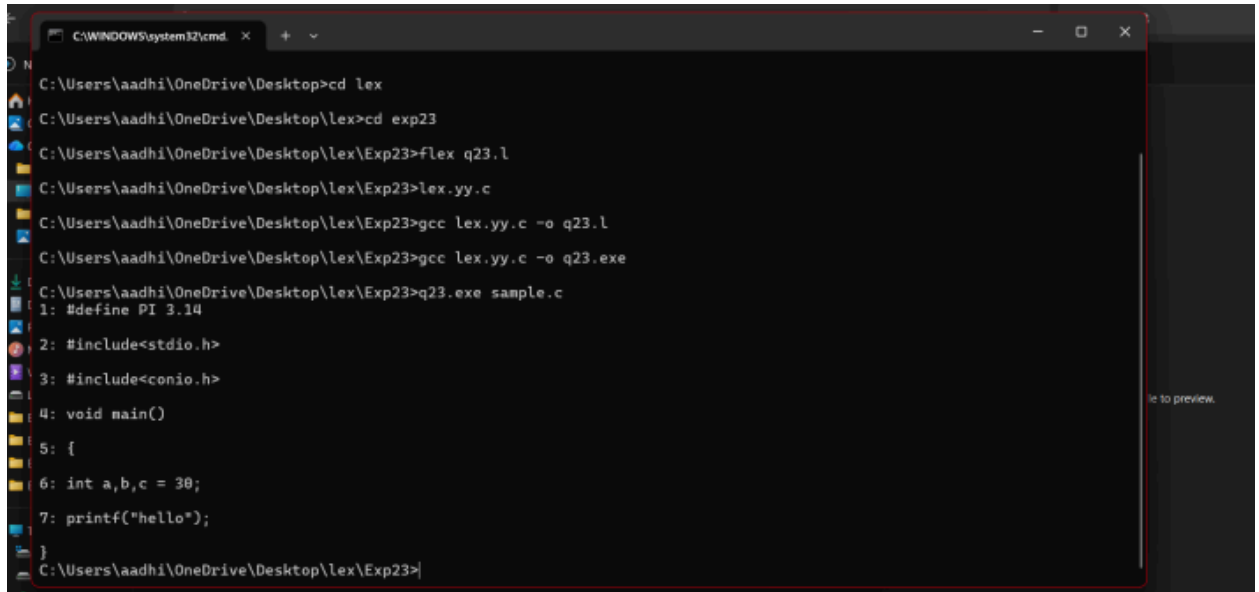
```
%{  
  
#include <stdio.h>  
  
int comment_lines = 0;  
  
FILE *out_file;  
  
%}  
  
  
%option noyywrap  
  
%%  
  
"//".*          { comment_lines++; }  
  
"/*"([^\*]|\\*+[^/*])*\*+"/{  
  
    for(int i=0; yytext[i] != '\0'; i++) {  
  
        if(yytext[i] == '\n') comment_lines++;  
  
    }  
  
    comment_lines++;  
  
}  
  
.|\\n          { fputs(yytext, out_file); }  
  
  
%%
```

```
int main(int argc, char **argv) {  
    if(argc > 1) {  
        yyin = fopen(argv[1], "r");  
    }  
    out_file = fopen("clean_output.c", "w");  
    yylex();  
    printf("Total comment lines eliminated: %d\n", comment_lines);  
    return 0;  
}
```

Sample Input:

```
#define PI 3.14  
  
#include<stdio.h>  
  
#include<conio.h>  
  
void main()  
{  
    int a,b,c = 30;  
    printf("hello");  
}
```

Output:



```
C:\WINDOWS\system32\cmd. X + v
C:\Users\aadhi\OneDrive\Desktop>cd lex
C:\Users\aadhi\OneDrive\Desktop\lex>cd exp23
C:\Users\aadhi\OneDrive\Desktop\lex\Exp23>flex q23.l
C:\Users\aadhi\OneDrive\Desktop\lex\Exp23>lex.yy.c
C:\Users\aadhi\OneDrive\Desktop\lex\Exp23>gcc lex.yy.c -o q23.l
C:\Users\aadhi\OneDrive\Desktop\lex\Exp23>gcc lex.yy.c -o q23.exe
C:\Users\aadhi\OneDrive\Desktop\lex\Exp23>q23.exe sample.c
1: #define PI 3.14
2: #include<stdio.h>
3: #include<conio.h>
4: void main()
5: {
6: int a,b,c = 30;
7: printf("hello");
}
C:\Users\aadhi\OneDrive\Desktop\lex\Exp23>
```

is to preview.

24. Write a LEX program to count the number of comment lines in a given C program and eliminate them and write into another file.

Input Source File: (input.c)

```
#include<stdio.h>

int main()

{

int a,b,c; /*variable declaration*/

printf("enter two numbers");

scanf("%d %d",&a,&b);

c=a+b;//adding two numbers

printf("sum is %d",c);

return 0;

}
```

Aim:

To write a LEX program to count and remove comments from a C program.

Short Procedure:

- 1. Read the input file.
- 2. Identify single-line and multi-line comments.
- 3. Count the comments.
- 4. Remove comments while copying.
- 5. Write the output to another file.

Code:

```
%{  
  
#include <stdio.h>  
  
int comment_count = 0;  
  
%}  
  
%%  
  
"/*"([^\]|\\*+[/\])*\\*+"/* { comment_count++; /* Match multi-line  
comments */ }  
  
"//".* { comment_count++; /* Match single-line  
comments */ }  
  
.|\\n { fprintf(yyout, "%s", yytext); /* Copy non-comment  
characters to output */ }
```

%%

```
int yywrap() {  
    return 1;  
}
```

```
int main(int argc, char *argv[]) {  
    if (argc < 3) {  
        printf("Usage: %s <input_file.c> <output_file.c>\n", argv[0]);  
        return 1;  
    }
```

```
    yyin = fopen(argv[1], "r");  
    if (!yyin) {  
        printf("Error: Cannot open input file %s\n", argv[1]);  
        return 1;  
    }
```

```
    yyout = fopen(argv[2], "w");
```

```
if (!yyout) {
```

```
    printf("Error: Cannot create output file %s\n", argv[2]);
```

```
    fclose(yyin);
```

```
    return 1;
```

```
}
```

```
yylex();
```

```
printf("Total comment lines eliminated: %d\n", comment_count);
```

```
fclose(yyin);
```

```
fclose(yyout);
```

```
return 0;
```

```
}
```

Sample Input:

```
#include<stdio.h>

int main()

{

int a,b,c; /*variable declaration*/

printf("enter two numbers");

scanf("%d %d",&a,&b);

c=a+b;//adding two numbers

printf("sum is %d",c);

return 0;

}
```

Output:

```
(c) Microsoft Corporation. All rights reserved.
C:\Users\aadhi>cd onedrive
C:\Users\aadhi\OneDrive>cd desktop
C:\Users\aadhi\OneDrive\Desktop>cd lex
C:\Users\aadhi\OneDrive\Desktop\lex>cd exp24
C:\Users\aadhi\OneDrive\Desktop\lex\Exp24>flex exp24.l
C:\Users\aadhi\OneDrive\Desktop\lex\Exp24>gcc lex.yy.c -o exp24
C:\Users\aadhi\OneDrive\Desktop\lex\Exp24>./exp24 sample.c output.c
'.' is not recognized as an internal or external command,
operable program or batch file.
C:\Users\aadhi\OneDrive\Desktop\lex\Exp24>./exp24 sample.c output.c
'.' is not recognized as an internal or external command,
operable program or batch file.
C:\Users\aadhi\OneDrive\Desktop\lex\Exp24>exp24.exe sample.c output.c
Total comment lines eliminated: 2
C:\Users\aadhi\OneDrive\Desktop\lex\Exp24>|
```

25. Write a LEX program to identify the capital words from the given input.

Aim:

To write a LEX program to identify and display capital words from the given input.

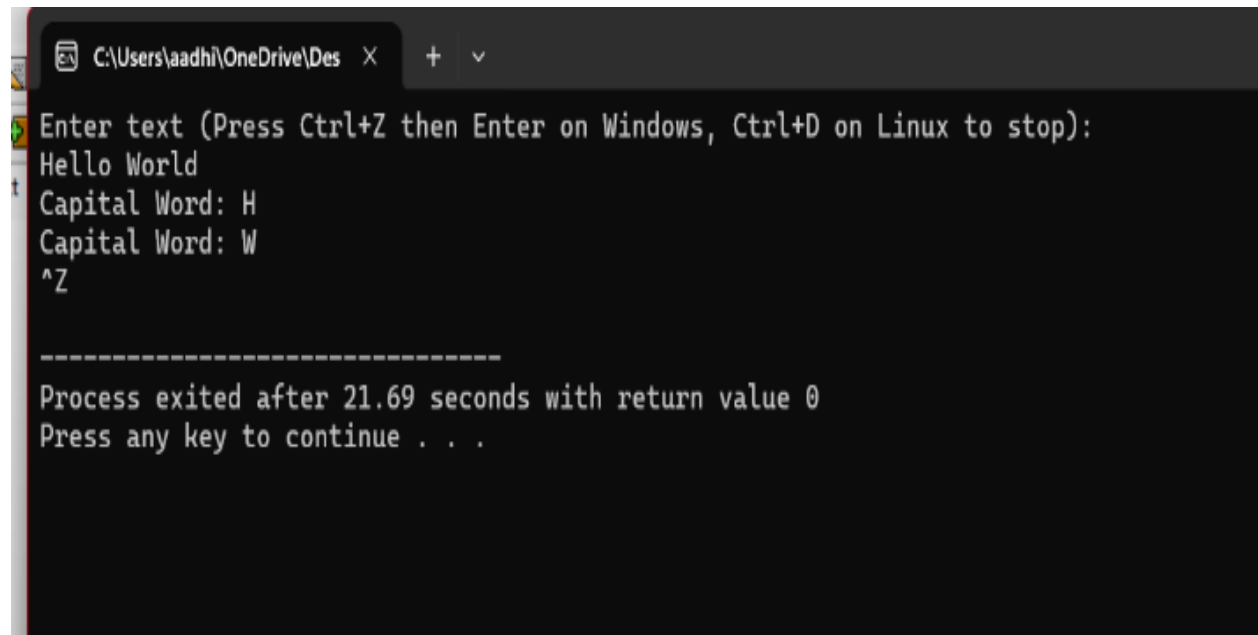
Short Procedure:

- 1. Create a LEX file with the .l extension.
- 2. Define a pattern to recognize words containing only capital letters (A–Z).
- 3. Write the corresponding action to display the identified capital words.
- 4. Save the LEX program and compile it using flex.
- 5. Compile the generated C file using gcc.
- 6. Run the program and enter the input text.
- 7. The program identifies and displays all the capital words from the input.

Code:

```
%{  
  
#include <stdio.h>  
  
%}  
  
%%  
  
[A-Z]+ { printf("Capital Word: %s\n", yytext); }  
.|\\n { /* Ignore all other characters */ }  
  
%%  
  
int yywrap() {  
    return 1;  
}  
  
int main() {  
    printf("Enter text (Press Ctrl+Z then Enter on Windows, Ctrl+D on  
Linux to stop):\n");  
    yylex();  
    return 0;  
}
```

Output:



```
C:\Users\aadhi\OneDrive\Desktop > .\Des.exe
Enter text (Press Ctrl+Z then Enter on Windows, Ctrl+D on Linux to stop):
Hello World
Capital Word: H
Capital Word: W
^Z

-----
Process exited after 21.69 seconds with return value 0
Press any key to continue . . .
```

The image shows a Windows command prompt window with a dark background. The title bar at the top indicates the file path 'C:\Users\aadhi\OneDrive\Desktop' and has standard window controls. The command prompt shows the execution of a program named 'Des.exe'. The program prompts the user to enter text, and the user enters 'Hello World'. The program then outputs 'Capital Word: H' and 'Capital Word: W'. The user then presses Ctrl+Z, which is represented as '^Z' in the output. A separator line of dashes follows. The program then displays 'Process exited after 21.69 seconds with return value 0' and 'Press any key to continue . . .', indicating the program has finished execution and is waiting for a key press to close the window.

26. Write a LEX Program to check the email address is valid or not.

Aim:

To write a LEX program to check whether a given email address is valid or not.

Short Procedure:

- ❖ 1. Create a LEX file with the .l extension.
- ❖ 2. Define a regular expression pattern to recognize a valid email structure (username@domain.extension).
- ❖ 3. Write the corresponding actions to set a flag or display whether the email is valid or invalid.
- ❖ 4. Save the LEX program and compile it using flex.
- ❖ 5. Compile the generated lex.yy.c file using gcc.
- ❖ 6. Run the executable and enter the input email address.
- ❖ 7. The program evaluates the input against the pattern and displays the result.

Code:

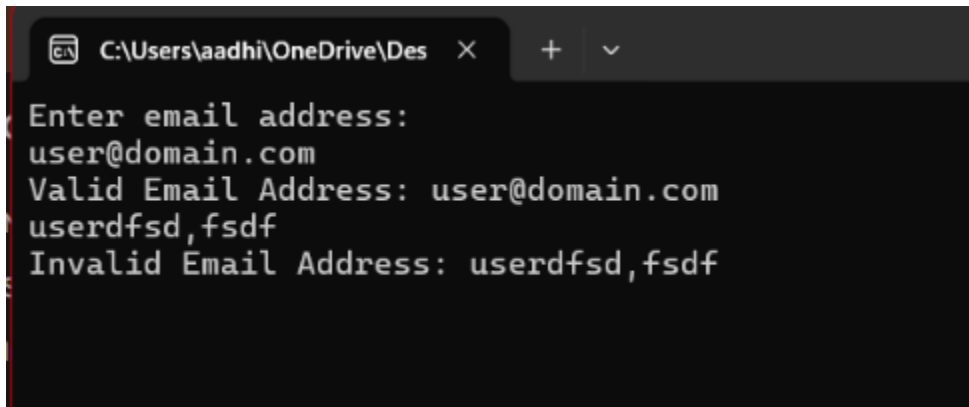
```
%{  
#include <stdio.h>  
%}  
  
%%  
[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,} { printf("Valid Email  
Address: %s\n", yytext); }  
.+ { printf("Invalid Email Address: %s\n",  
yytext); }
```

```
\n                                { /* Ignore newlines */ }  
%%
```

```
int yywrap() {  
    return 1;  
}
```

```
int main() {  
    printf("Enter email address:\n");  
    yylex();  
    return 0;  
}
```

Output:



```
C:\Users\aadhi\OneDrive\Des  X + v  
Enter email address:  
user@domain.com  
Valid Email Address: user@domain.com  
userdfsd,fsdf  
Invalid Email Address: userdfsd,fsdf
```

27. Write a LEX Program to convert the substring abc to ABC from the given input string.

Aim:

To write a LEX program that converts all occurrences of the substring abc to ABC in a given input string.

Short Procedure:

- 1. Create a LEX file with the .l extension.
- 2. Define a pattern to match the specific substring abc.
- 3. Write an action for the pattern abc to print ABC.
- 4. Include a default rule (.|\\n) to echo all other characters without changes.
- 5. Save the LEX file and compile it using flex.
- 6. Compile the generated lex.yy.c file using gcc.
- 7. Run the executable, enter the input string, and observe the modified output.

Code:

```
%{
#include <stdio.h>
}%

%%

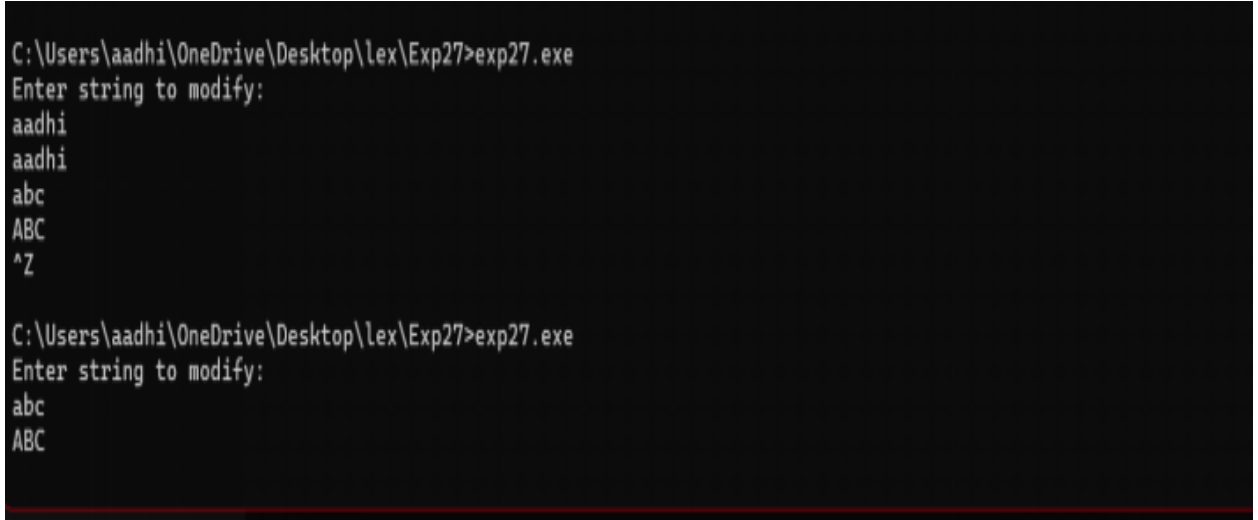
"abc"  { printf("ABC"); }
.|\\n  { printf("%s", yytext); }
%%

int yywrap() {
```

```
    return 1;
}

int main() {
    printf("Enter string to modify:\n");
    yylex();
    return 0;
}
```

Output:



```
C:\Users\aadhi\OneDrive\Desktop\lex\Exp27>exp27.exe
Enter string to modify:
aadhi
aadhi
abc
ABC
^Z

C:\Users\aadhi\OneDrive\Desktop\lex\Exp27>exp27.exe
Enter string to modify:
abc
ABC
```

28. The Company ABC runs with employees with several departments. The Organization manager had all the mobile numbers of employees. Assume that you are the manager and need to verify the valid mobile numbers because there may be some invalid numbers present. Implement a LEX program to check whether the mobile number is valid or not.

Aim:

To write a LEX program to check whether a given mobile number is valid or invalid.

Short Procedure:

- Create a LEX file with a pattern for a valid mobile number.
- Accept the mobile number as input.
- Check whether it contains exactly 10 digits and starts with 6, 7, 8, or 9
- Display whether the mobile number is valid or invalid.
- Compile and execute the LEX program.

Code:

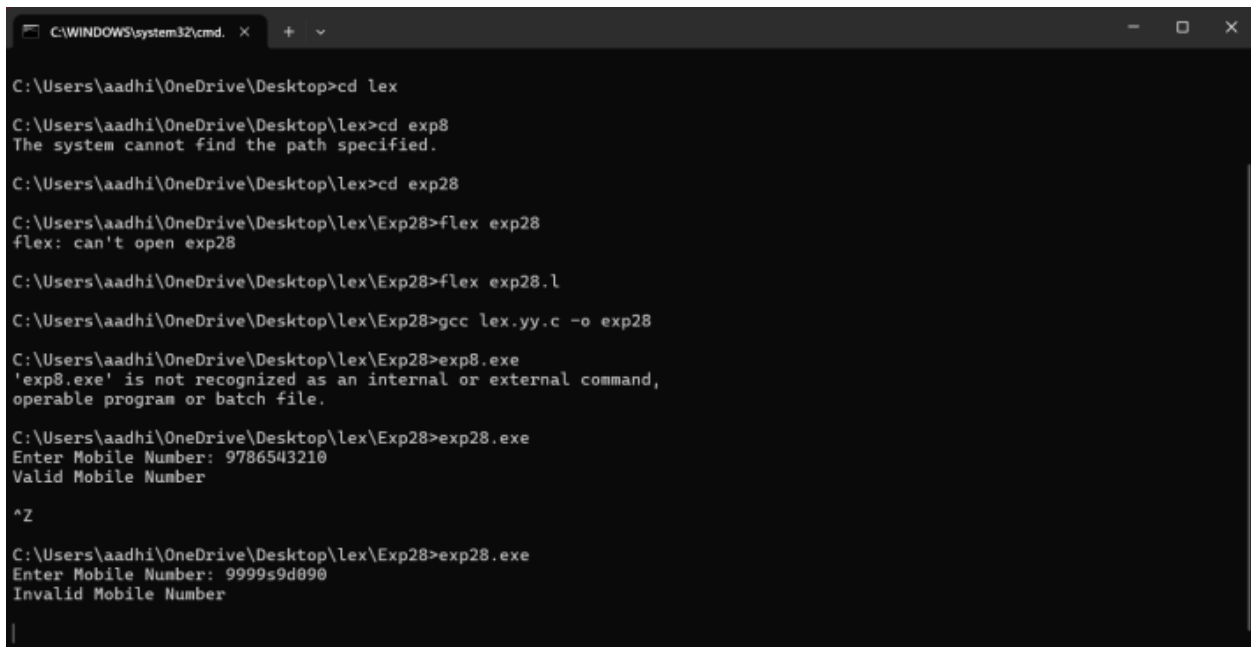
```
%{
#include <stdio.h>
}%

%%
[6-9][0-9]{9}    { printf("Valid Mobile Number\n"); }
[0-9]+          { printf("Invalid Mobile Number\n"); }
.*              { printf("Invalid Mobile Number\n"); }
%%

int yywrap()
{
    return 1;
}
```

```
int main()
{
    printf("Enter Mobile Number: ");
    yylex();
    return 0;
}
```

OUTPUT



```
C:\WINDOWS\system32\cmd. X + v
C:\Users\aadhi\OneDrive\Desktop>cd lex
C:\Users\aadhi\OneDrive\Desktop\lex>cd exp8
The system cannot find the path specified.
C:\Users\aadhi\OneDrive\Desktop\lex>cd exp28
C:\Users\aadhi\OneDrive\Desktop\lex\Exp28>flex exp28
flex: can't open exp28
C:\Users\aadhi\OneDrive\Desktop\lex\Exp28>flex exp28.l
C:\Users\aadhi\OneDrive\Desktop\lex\Exp28>gcc lex.yy.c -o exp28
C:\Users\aadhi\OneDrive\Desktop\lex\Exp28>exp28.exe
'exp28.exe' is not recognized as an internal or external command,
operable program or batch file.
C:\Users\aadhi\OneDrive\Desktop\lex\Exp28>exp28.exe
Enter Mobile Number: 9786543210
Valid Mobile Number
^Z
C:\Users\aadhi\OneDrive\Desktop\lex\Exp28>exp28.exe
Enter Mobile Number: 9999s9d090
Invalid Mobile Number
|
```

29. Implement Lexical Analyzer using LEX or FLEX (Fast Lexical Analyzer). The program should separate the tokens in the given C program and display them with the appropriate caption.

Aim:

To implement a lexical analyzer using LEX/FLEX to identify and display different tokens present in a C program with appropriate captions.

Short Procedure:

- Create a LEX program containing patterns for keywords, identifiers, numbers, operators, strings, and special symbols.
- Give the C source program as input.
- The lexical analyzer scans the source program character by character.
- It identifies each token and displays its corresponding type.
- Compile and execute the program using FLEX and GCC.

Code:

```
%{
#include <stdio.h>
}%

%%

#include<stdio.h>      { printf("PREPROCESSOR : %s\n", yytext); }

"void"|"int"|"char"|"float" { printf("KEYWORD      : %s\n", yytext); }

"main"                { printf("FUNCTION       : %s\n", yytext); }

[a-zA-Z_][a-zA-Z0-9_]*  { printf("IDENTIFIER   : %s\n", yytext); }

[0-9]+                 { printf("NUMBER        : %s\n", yytext); }
```

```

\[^\"]*"          { printf("STRING      : %s\n", yytext); }

"="|"+"|"-"|"*"|"|"  { printf("OPERATOR    : %s\n", yytext); }

"(" | ")" | "{" | "}" | ";" | ","
                    { printf("SPECIAL SYMBOL: %s\n", yytext); }

[ \t\n]+          { /* Ignore whitespace */ }

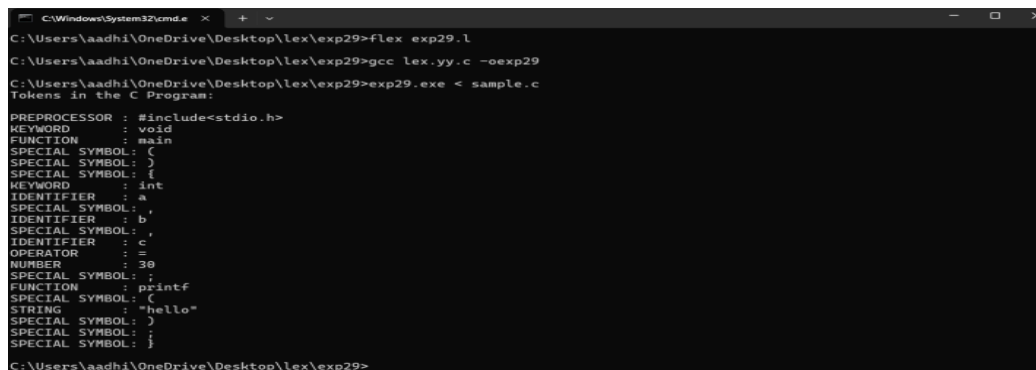
.                  { printf("UNKNOWN      : %s\n", yytext); }
%%

int yywrap()
{
    return 1;
}

int main()
{
    printf("Tokens in the C Program:\n\n");
    yylex();
    return 0;
}

```

Output:



```

C:\Windows\System32\cmd.exe
C:\Users\aadhi\OneDrive\Desktop\lex\exp29>flex exp29.l
C:\Users\aadhi\OneDrive\Desktop\lex\exp29>gcc lex.yy.c -oexp29
C:\Users\aadhi\OneDrive\Desktop\lex\exp29>exp29.exe < sample.c
Tokens in the C Program:
PREPROCESSOR : #include<stdio.h>
KEYWORD      : void
FUNCTION     : main
SPECIAL SYMBOL: (
SPECIAL SYMBOL: )
SPECIAL SYMBOL: {
KEYWORD      : int
IDENTIFIER   : a
SPECIAL SYMBOL: ,
IDENTIFIER   : b
SPECIAL SYMBOL: ,
IDENTIFIER   : c
OPERATOR     : =
NUMBER       : 30
SPECIAL SYMBOL: ;
FUNCTION     : printf
SPECIAL SYMBOL: (
STRING       : "hello"
SPECIAL SYMBOL: )
SPECIAL SYMBOL: ;
SPECIAL SYMBOL: }
C:\Users\aadhi\OneDrive\Desktop\lex\exp29>

```

30.) In a class, an English teacher was teaching the vowels and consonants to the students. She says “Vowel sounds allow the air to flow freely, causing the chin to drop noticeably, whilst consonant sounds are produced by restricting the airflow”. As a class activity, the students are asked to identify the vowels and consonants in the given word/sentence and count the number of elements in each. Write an algorithm to help the student to count the number of consonants in the given sentence.

Aim:

To write a LEX program to identify and count the number of consonants present in a given word or sentence.

Short Procedure:

- Create a LEX program with patterns for alphabetic characters.
- Check each character in the input.
- If the character is an alphabet and is not a vowel, count it as a consonant.
- Ignore spaces, digits, and special characters.
- Display the total number of consonants.

Code :

```
%{  
#include <stdio.h>  
  
int consonants = 0;  
%}  
  
%%  
  
[a-zA-Z] {
```

```

    if (yytext[0] != 'a' && yytext[0] != 'e' &&
        yytext[0] != 'i' && yytext[0] != 'o' &&
        yytext[0] != 'u' &&
        yytext[0] != 'A' && yytext[0] != 'E' &&
        yytext[0] != 'I' && yytext[0] != 'O' &&
        yytext[0] != 'U')
    {
        consonants++;
    }
}

\n {
    printf("\nNumber of consonants = %d\n", consonants);
    return 0;
}

.;

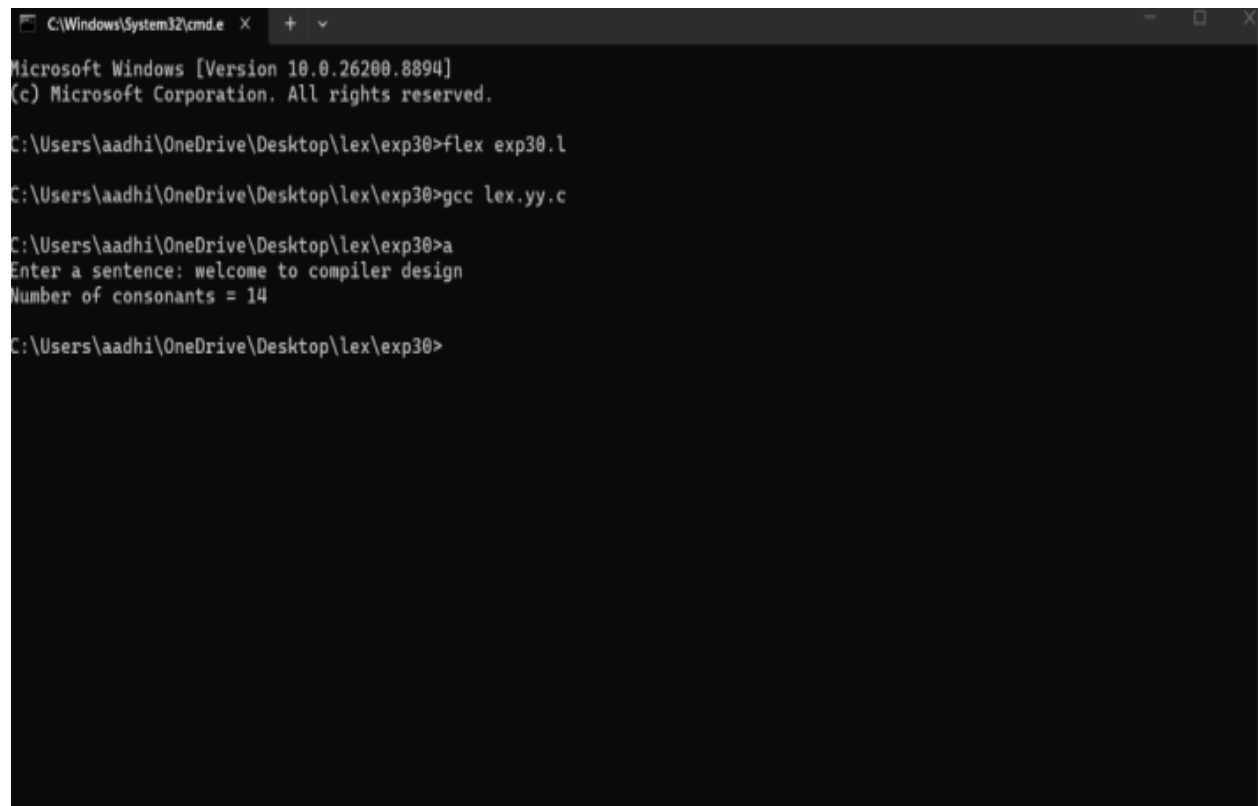
%%

int yywrap()
{
    return 1;
}

int main()
{
    printf("Enter a sentence: ");
    yylex();
    return 0;
}

```

OUTPUT



```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.26200.8894]
(c) Microsoft Corporation. All rights reserved.

C:\Users\aadhi\OneDrive\Desktop\lex\exp30>flex exp30.l

C:\Users\aadhi\OneDrive\Desktop\lex\exp30>gcc lex.yy.c

C:\Users\aadhi\OneDrive\Desktop\lex\exp30>a
Enter a sentence: welcome to compiler design
Number of consonants = 14

C:\Users\aadhi\OneDrive\Desktop\lex\exp30>
```

31. Keywords are predefined, reserved words used in programming that have special meanings to the compiler. Keywords are part of the syntax and they cannot be used as an identifier. In general, there are 32 keywords. The prime function of Lexical Analyser is token Generation. Among the 6 types of tokens, differentiating Keywords and identifiers is a challenging issue. Thus write a LEX program to separate keywords and identifiers.

Aim:

To write a LEX program to separate keywords and identifiers from the given input and generate the corresponding tokens.

Short Procedure:

- ❖ 1. Define the keywords and identifier pattern in the LEX program.
- ❖ 2. Read the input source code character by character.
- ❖ 3. Compare each word with the predefined list of keywords.
- ❖ 4. If the word matches a keyword, classify it as a keyword; otherwise, classify it as an identifier.
- ❖ 5. Compile and execute the LEX program to display the separated keywords and identifiers.

Code:

```
%{
#include <stdio.h>
%}

%%

"auto"|"break"|"case"|"char"|"const"|"continue"|"default"|"do"|"double"|"el
se"|"enum"|"extern"
"float"|"for"|"goto"|"if"|"int"|"long"|"register"|"return"|"short"|"signed"|"siz
eof"|"static"|"struct"
|"switch"|"typedef"|"union"|"unsigned"|"void"|"volatile"|"while"    {
printf("Keyword : %s\n", yytext); }

[a-zA-Z_][a-zA-Z0-9_]*          {

printf("Identifier : %s\n", yytext); } [

\t\n]+ ;

. ;

%%

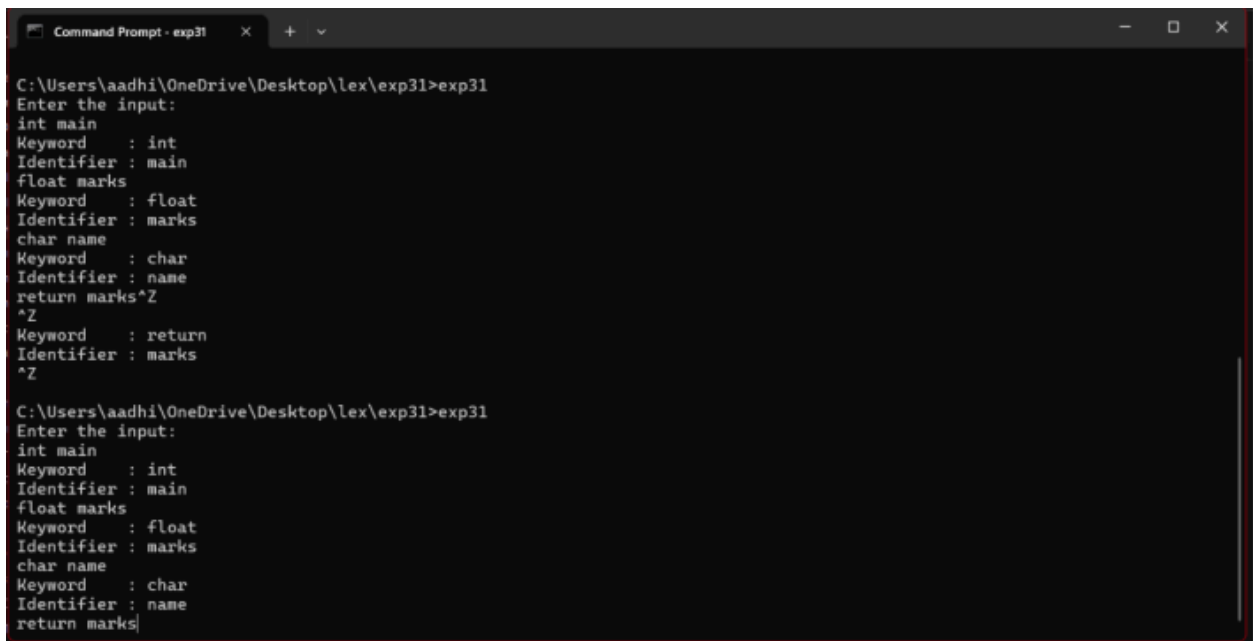
int main()
{
    printf("E
nter the
input:\n"
);
    yylex();
    return 0;
```

```
}  
  
int yywrap()  
{  
    return 1;  
}
```

Sample Input:

Int main
Float Marks
Char name
Return marks

OUTPUT



```
Command Prompt - exp31  
C:\Users\aadhi\OneDrive\Desktop\lex\exp31>exp31  
Enter the input:  
int main  
Keyword   : int  
Identifier : main  
float marks  
Keyword   : float  
Identifier : marks  
char name  
Keyword   : char  
Identifier : name  
return marks^Z  
^Z  
Keyword   : return  
Identifier : marks  
^Z  
  
C:\Users\aadhi\OneDrive\Desktop\lex\exp31>exp31  
Enter the input:  
int main  
Keyword   : int  
Identifier : main  
float marks  
Keyword   : float  
Identifier : marks  
char name  
Keyword   : char  
Identifier : name  
return marks
```

32. Write a LEX program to identify and count positive and negative numbers.

Aim:

To write a LEX program to identify and count positive and negative numbers from the given input.

Short Procedure:

- 1. Define patterns for positive and negative numbers.
- 2. Read the input using the LEX analyzer.
- 3. Identify numbers beginning with + or without a sign as positive.
- 4. Identify numbers beginning with - as negative.
- 5. Increment the respective counters and display the total count.

Code:

```
%{  
#include <stdio.h>  
Int positive = 0 ;  
Int Negative = 0;  
%}  
  
%%  
  
[+]?[0-9]+(\\.[0-9]+)? {  
    printf("Positive Number :  
    %s\\n", yytext);  
    positive++;
```

```
}
```

```
-[0-9]+(\\.[0-9]+)? {  
    printf("Negative Number :  
    %s\\n", yytext);  
    negative++;  
}
```

```
[ \\t\\n]+ ;
```

```
. ;
```

```
%%
```

```
int main()  
{  
    printf("Enter  
    numbers:  
    \\n");  
    yylex();  
  
    printf("\\nPositive Numbers Count =  
    %d\\n", positive); printf("Negative  
    Numbers Count = %d\\n", negative);  
  
    return 0;  
}
```

```
int yywrap()  
{  
    return 1;  
}
```

Sample Input:

10 -20 30 40 -1 -55

Output:

```
C:\WINDOWS\system32\cmd. X + v
Microsoft Windows [Version 10.0.26200.8894]
(c) Microsoft Corporation. All rights reserved.

C:\Users\aadhi>cd onedrive
C:\Users\aadhi\OneDrive>cd desktop
C:\Users\aadhi\OneDrive\Desktop>cd lex
C:\Users\aadhi\OneDrive\Desktop\lex>cd exp32
C:\Users\aadhi\OneDrive\Desktop\lex\exp32>flex exp32.l
C:\Users\aadhi\OneDrive\Desktop\lex\exp32>gcc lex.yy.c exp32
gcc: error: exp32: No such file or directory
C:\Users\aadhi\OneDrive\Desktop\lex\exp32>lex.yy.c
C:\Users\aadhi\OneDrive\Desktop\lex\exp32>gcc lex.yy.c -o exp32
C:\Users\aadhi\OneDrive\Desktop\lex\exp32>exp32
Enter numbers:
10 -20 30 40 -1 -55
Positive Number : 10
Negative Number : -20
Positive Number : 30
Positive Number : 40
Negative Number : -1
Negative Number : -55
```

33. A networking company wants to validate the URL for its clients. Write a LEX program to implement the same.

Aim:

To write a LEX program to validate a URL and determine whether the given URL is valid or invalid.

Short Procedure:

- Define a regular expression for a valid URL.
- Read the URL using the LEX analyzer.
- Match the input against the URL pattern.
- If it matches, display **Valid URL**.
- Otherwise, display **Invalid URL**.

Code

```
%{
#include <stdio.h>
}%

%%

https?://[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}(/[^\t\n]*)? {

    printf("Valid URL : %s\n", yytext);
}

[^\n]+ {
    printf("Invalid URL : %s\n", yytext);
}
```

```
\n ;

%%

int main()
{

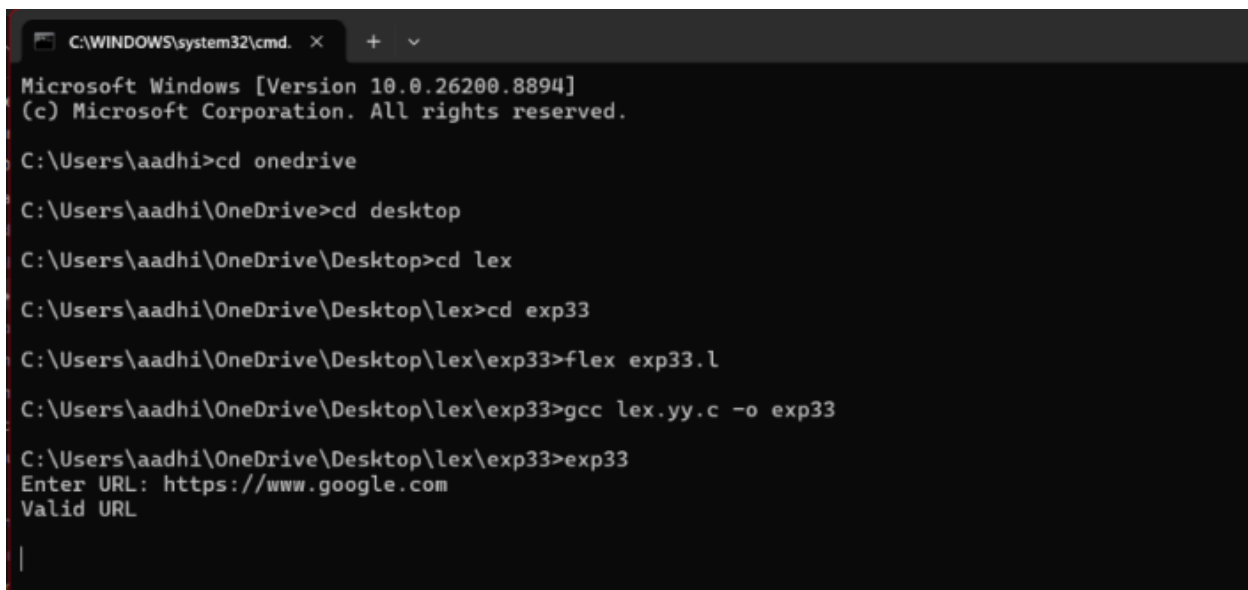
    return 0;
}

int yywrap()
{
    return 1;
}
```

Sample Input:

<https://www.google.com>

Output:



```
C:\WINDOWS\system32\cmd. x + v
Microsoft Windows [Version 10.0.26200.8894]
(c) Microsoft Corporation. All rights reserved.

C:\Users\aadhi>cd onedrive
C:\Users\aadhi\OneDrive>cd desktop
C:\Users\aadhi\OneDrive\Desktop>cd lex
C:\Users\aadhi\OneDrive\Desktop\lex>cd exp33
C:\Users\aadhi\OneDrive\Desktop\lex\exp33>flex exp33.l
C:\Users\aadhi\OneDrive\Desktop\lex\exp33>gcc lex.yy.c -o exp33
C:\Users\aadhi\OneDrive\Desktop\lex\exp33>exp33
Enter URL: https://www.google.com
Valid URL
|
```

**34. School management wants to validate the DOB of all students.
Write a LEX program to implement it.**

Aim:

To write a LEX program to validate the Date of Birth (DOB) of students in the format DD/MM/YYYY.

Short procedure:

- Define a regular expression for the DOB format DD/MM/YYYY.
- Accept the day from 01–31.
- Accept the month from 01–12.
- Accept a four-digit year.
- If the input matches the pattern, display Valid DOB; otherwise display Invalid DOB.

Code:

```
%{  
#include <stdio.h>  
%}  
  
%%  
  
(0[1-9]|[12][0-9]|3[01])[/](0[1-9]|1[0-2])[/][0-9]{4} { printf("Valid DOB\n"); }  
  
.* { printf("Invalid DOB\n"); }  
  
%%  
  
int main()  
{
```

```

        printf("Enter   DOB
        (DD/MM/YYYY):
        "); yylex();
        return 0;
    }

    int yywrap()
    {
        return 1;
    }

```

Sample Input:

27/10/2007

OUTPUT

```

C:\WINDOWS\system32\cmd. X + v
Microsoft Windows [Version 10.0.26200.8894]
(c) Microsoft Corporation. All rights reserved.

C:\Users\aadhi>cd onedrive
C:\Users\aadhi\OneDrive>cd desktop
C:\Users\aadhi\OneDrive\Desktop>cd lex
C:\Users\aadhi\OneDrive\Desktop\lex>cd exp34
C:\Users\aadhi\OneDrive\Desktop\lex\exp34>flex exp34.l
C:\Users\aadhi\OneDrive\Desktop\lex\exp34>gcc lex.yy.c -o exp34
C:\Users\aadhi\OneDrive\Desktop\lex\exp34>exp34
Enter DOB (DD/MM/YYYY): 27/10/2007
Valid DOB

^Z

C:\Users\aadhi\OneDrive\Desktop\lex\exp34>exp34
Enter DOB (DD/MM/YYYY): 2007/10/27
Invalid DOB
|

```

35. Write a LEX program to check whether the given input is a digit or not.

Aim:

To write a LEX program to check whether the given input is a digit or not.

Short Procedure:

- 1. Define the pattern [0-9] to recognize a digit.
- 2. Read the input character using LEX.
- 3. If the character matches [0-9], display It is a Digit.
- 4. Otherwise, display It is Not a Digit.

Code:

```
%{  
#include <stdio.h>  
%}  
  
%%  
  
[0-9] { printf("It is a Digit\n"); }  
  
. { printf("It is Not a Digit\n"); }  
  
%%  
  
int main()  
{  
    printf("Enter a  
character:  
");  
    yylex();  
    return 0;  
}
```

```
int yywrap()
{
    return 1;
}
```

A screenshot of a Windows Command Prompt window titled "C:\WINDOWS\system32\cmd". The window shows the following sequence of commands and outputs:

```
C:\Users\aadhi\OneDrive\Desktop\lex\exp35>flex exp35.l  
C:\Users\aadhi\OneDrive\Desktop\lex\exp35>gcc lex.yy.c -o exp35  
C:\Users\aadhi\OneDrive\Desktop\lex\exp35>exp35  
Enter a character: 47487484  
It is a Digit  
It is a Digit  
It is a Digit  
It is a Digit  
It is a Digit  
It is a Digit  
It is a Digit  
It is a Digit  
It is a Digit  
^Z  
C:\Users\aadhi\OneDrive\Desktop\lex\exp35>exp35  
Enter a character: fkdjfkwdjf  
It is Not a Digit  
It is Not a Digit  
It is Not a Digit  
It is Not a Digit  
It is Not a Digit  
It is Not a Digit  
It is Not a Digit  
It is Not a Digit  
It is Not a Digit  
It is Not a Digit
```

The first part demonstrates successful digit recognition for numeric input. After pressing Ctrl-Z (^Z), the second part shows that non-digit characters are correctly identified as "Not a Digit".

36. A School student was asked to do basic mathematical operations. Implement a LEX program to implement the same.

Aim:

To write a LEX program to perform basic mathematical operations such as addition, subtraction, multiplication, and division.

Short Procedure:

- 1. Define patterns for numbers and mathematical operators.
- 2. Read the arithmetic expression using LEX.
- 3. Identify the operator (+, -, *, /).
- 4. Perform the corresponding mathematical operation.
- 5. Display the calculated result.

Code:

```
%%
```

```
[0-9]+[ \t]*[+|-|*|/][ \t]*[0-9]+ {  
    sscanf(yytext, "%d %c %d", &num1, &op, &num2);
```

```
    if(op == '+')  
        printf("Result    =  
%d\n",    num1    +  
num2); else if(op ==  
'-')  
        printf("Result    =  
%d\n",    num1    -  
num2); else if(op ==  
'*')  
        printf("Result    =  
%d\n",    num1    *
```

```

num2); else if(op ==
 '/')
{
    if(num2 != 0)
        printf("Result    =
%d\n",    num1    /
num2); else
    printf("Cannot divide by zero\n");
}
}

```

```

[ \t\n]+ ;

```

```

. {
    printf("Invalid Input\n");
}

```

```

%%

```

```

int main()
{
    printf("E
nter
expressio
n:    ");
    yylex();
    return 0;
}

```

```

int yywrap()
{
    return 1;
}

```

Output:

```
C:\WINDOWS\system32\cmd. x + v
C:\Users\aadhi\OneDrive\Desktop\lex>cd exp36
C:\Users\aadhi\OneDrive\Desktop\lex\exp36>flex exp36.l
C:\Users\aadhi\OneDrive\Desktop\lex\exp36>gcc lex.yy.c -o exp36
C:\Users\aadhi\OneDrive\Desktop\lex\exp36>exp36
Enter expression: 10+5
Result = 15
^Z

C:\Users\aadhi\OneDrive\Desktop\lex\exp36>exp36
Enter expression: 20-8
Result = 12
^Z

C:\Users\aadhi\OneDrive\Desktop\lex\exp36>exp36
Enter expression: 3*8
Result = 24
^Z

C:\Users\aadhi\OneDrive\Desktop\lex\exp36>exp38
'exp38' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\aadhi\OneDrive\Desktop\lex\exp36>exp36
Enter expression: 12/4
Result = 3
|
```

37. Write a Lex program to count the frequency of the given word in a given sentence.

Aim:

To write a LEX program to count the frequency of a given word in a sentence.

Short Procedure:

- 1. Read the word whose frequency needs to be counted.
- 2. Read the sentence using LEX.
- 3. Identify each word from the sentence.
- 4. Compare each word with the given word.
- 5. Increment the counter whenever a match is found and display the frequency.

Code:

```
%%
```

```
[a-zA-Z]+ {  
    if(strc  
        mp(  
        yyte  
        xt,  
        wor  
        d)  
        ==  
        0)  
        cou
```

```

        nt++;
        ;
    }

[ \t\n]+ ;

. ;

%%

int main()
{
    printf("Enter the
word to search:
");    scanf("%s",
word);

    printf("E
nter the
sentence:
\n");
    getchar();

    yylex();

    printf("Frequency of '%s' = %d\n", word, count);

    return 0;
}

int yywrap()
{

    return 1;
}

```

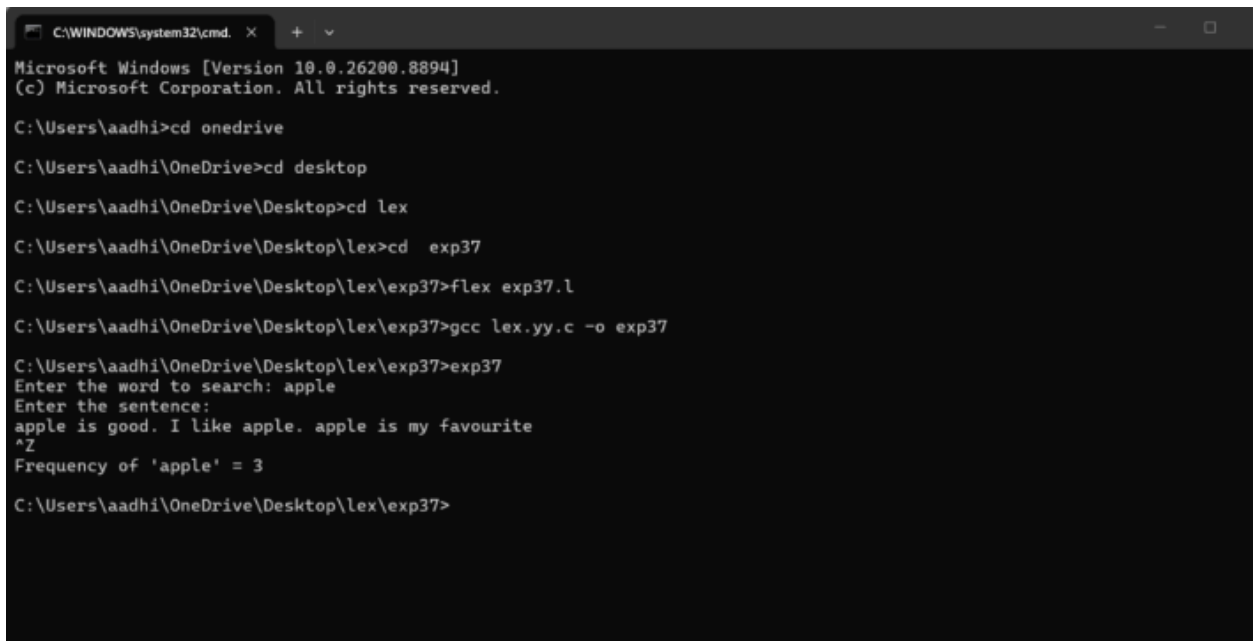
}

Sample Input:

Apple.

Apple is good. I like apple. Apple is my favourite.

Output:



```
C:\WINDOWS\system32\cmd. X + v
Microsoft Windows [Version 10.0.26200.8894]
(c) Microsoft Corporation. All rights reserved.

C:\Users\aadhi>cd onedrive
C:\Users\aadhi\OneDrive>cd desktop
C:\Users\aadhi\OneDrive\Desktop>cd lex
C:\Users\aadhi\OneDrive\Desktop\lex>cd exp37
C:\Users\aadhi\OneDrive\Desktop\lex\exp37>flex exp37.l
C:\Users\aadhi\OneDrive\Desktop\lex\exp37>gcc lex.yy.c -o exp37
C:\Users\aadhi\OneDrive\Desktop\lex\exp37>exp37
Enter the word to search: apple
Enter the sentence:
apple is good. I like apple. apple is my favourite
^Z
Frequency of 'apple' = 3
C:\Users\aadhi\OneDrive\Desktop\lex\exp37>
```

38. Write a Lex program to find the length of the longest word.

Aim:

To write a LEX program to find the length of the longest word in a given sentence.

Short Procedure:

1. Read the sentence using LEX.
2. Identify each word using [a-zA-Z]+.
3. Find the length of each word using strlen().
4. Compare each word length with the current maximum.
5. Store and display the longest word and its length.

Code:

```
#include
<stdio.h>
#include
<string.h>
%{
    int max = 0;
    char longest[100];
}%
%%

[a-zA-Z]+ {
    if(strlen(yytext) > max)
    {
```

```

        max =
        strlen(
        yytext)
        ;
        strcpy(
        longest
        ,
        yytext)
        ;
    }
}

```

```

[ \t\n]+ ;

```

```

. ;

```

```

%%

```

```

int main()
{
    printf("E
    nter a
    sentence:
    \n");
    yylex();

    printf("Longest word =
    %s\n", longest);
    printf("Length of longest
    word = %d\n", max);

    return 0;
}

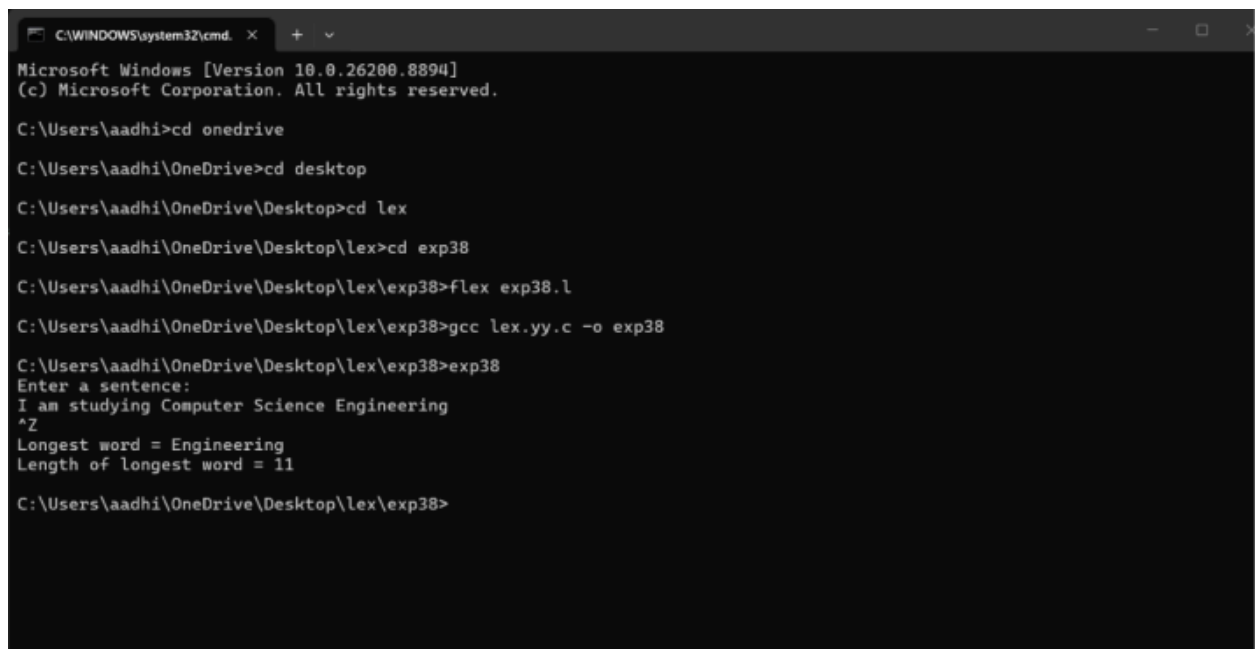
```

```
int yywrap()  
{  
    return 1;  
}
```

Sample Input:

I am studying in Computer Science Engineering

OUTPUT



```
C:\WINDOWS\system32\cmd. x + v  
Microsoft Windows [Version 10.0.26200.8894]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Users\aadhi>cd onedrive  
C:\Users\aadhi\OneDrive>cd desktop  
C:\Users\aadhi\OneDrive\Desktop>cd lex  
C:\Users\aadhi\OneDrive\Desktop\lex>cd exp38  
C:\Users\aadhi\OneDrive\Desktop\lex\exp38>flex exp38.l  
C:\Users\aadhi\OneDrive\Desktop\lex\exp38>gcc lex.yy.c -o exp38  
C:\Users\aadhi\OneDrive\Desktop\lex\exp38>exp38  
Enter a sentence:  
I am studying Computer Science Engineering  
^Z  
Longest word = Engineering  
Length of longest word = 11  
C:\Users\aadhi\OneDrive\Desktop\lex\exp38>
```

31. Write a Lex code to replace a word with another word in a file.

Aim:

To write a LEX program to replace a given word with another word in a file.

Short Procedure:

- 1. Read **the old word and the new word.**
- 2. Read **the file content using LEX.**
- 3. Compare **each word with the old word.**
- 4. If **it matches, print the new word.**
- 5. Otherwise, **print the original word.**

Code:

```
%{  
#include  
<stdio.h>  
#include  
<string.h>  
  
char  
oldwo  
rd[100  
]; char  
newwo  
rd[100  
];  
%}
```

%%

[a-zA-Z]+ {

```
if(strcmp(yytext, oldword)
    == 0)
    printf("%s",
        ",
        newword);
```

else

```
printf("%s", yytext);
}
```

[\\t\\n]+ {

```
printf("%s", yytext);
}
```

. {

```
printf("%s", yytext);
}
```

%%

int main()

```
{
printf("Enter word to
    replace: ");
scanf("%s",
    oldword);
```

```
printf("Enter new
    word:
    ");
scanf("
    %s",
```

```
        newword
    );

    printf("\nEnter the file
        content:\n");

    yylex();

return 0;
}
```

```
int yywrap()
{
return 1;
}
```

Sample Input:

I like apple. Mango is very tasty.

Output:

```
C:\WINDOWS\system32\cmd. x + v
Microsoft Windows [Version 10.0.26200.8894]
(c) Microsoft Corporation. All rights reserved.

C:\Users\aadhi>cd onedrive
C:\Users\aadhi\OneDrive>cd desktop
C:\Users\aadhi\OneDrive\Desktop>cd lex
C:\Users\aadhi\OneDrive\Desktop\lex>cd xp39
The system cannot find the path specified.
C:\Users\aadhi\OneDrive\Desktop\lex>cd exp39
C:\Users\aadhi\OneDrive\Desktop\lex\exp39>flex exp39.l
C:\Users\aadhi\OneDrive\Desktop\lex\exp39>gcc lex.yy.c
C:\Users\aadhi\OneDrive\Desktop\lex\exp39>gcc lex.yy.c -o exp39
C:\Users\aadhi\OneDrive\Desktop\lex\exp39>exp39
Enter word to replace: apple
Enter new word: mango

Enter the file content:
I like apple. Mango is very tasty

I like mango. Mango is very tasty
```

40 . Write a FLEX (Fast Lexical Analyzer) program that should perform the token separation for the given C program and display with appropriate caption.

Aim:

To write a FLEX program to perform token separation for a given C program and display the tokens with appropriate captions such as Keywords, Identifiers, Numbers, Operators, Special Symbols, and String Literals.

Short Procedure:

- Define regular expressions for different C tokens.
- Read the given C program using FLEX.
- Identify keywords, identifiers, numbers, operators, strings, and special symbols.
- Display each token with its appropriate caption.
- Ignore spaces, newlines, and comments.

Code:

```
%{  
#include <stdio.h>  
%}  
  
%%  
  
"int"|"float"|"char"|"double"|"if"|"else"|"for"|"while"|"return"|"void"|"break"|"continue" { printf("KEYWORD : %s\n", yytext); }
```

```

[a-zA-Z_][a-zA-Z0-9_]* {
printf("IDENTIFIER : %s\n", yytext); } [0-9]+
{ printf("NUMBER : %s\n", yytext); }
"=="|"!="|"<="|">="|"++"|--" { printf("OPERATOR : %s\n", yytext); }
"+"|"-"|"*"|"/"|"="|"<"|>"|"%" { printf("OPERATOR : %s\n", yytext); }
"("|")"|"{"|"}"|"["|"]"|"|";|", " { printf("SPECIAL
SYMBOL : %s\n", yytext); } [ \t\n]+ ;
. { printf("UNKNOWN : %s\n", yytext); }

%%

int main()
{
printf("TOKEN
SEPARATION\n
");
printf("=====  

=====\\n");

}

int yywrap()
{
return 1;
}

```

OUTPUT

```
C:\WINDOWS\system32\cmd. x + v
TOKEN SEPARATION
=====
int main()
KEYWORD      : int
IDENTIFIER   : main
SPECIAL SYMBOL : (
SPECIAL SYMBOL : )
{
SPECIAL SYMBOL : {
    int a = 10;
KEYWORD      : int
IDENTIFIER   : a
OPERATOR     : =
NUMBER      : 10
SPECIAL SYMBOL : ;
    int b = 20;
KEYWORD      : int
IDENTIFIER   : b
OPERATOR     : =
NUMBER      : 20
SPECIAL SYMBOL : ;
    a = a + b;
IDENTIFIER   : a
OPERATOR     : =
IDENTIFIER   : a
OPERATOR     : +
IDENTIFIER   : b
SPECIAL SYMBOL : ;
    return 0;
KEYWORD      : return
```